HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN I

BÁO CÁO HỌC PHẦN

HỌC PHẦN: IOT VÀ ỨNG DỤNG

ĐỀ TÀI:

Hệ thống giám sát nhiệt độ, độ ẩm và ánh sáng môi trường phòng học sử dụng ESP32

Giảng viên: Nguyễn Quốc Uy

Sinh viên: Hoàng Mạnh Hiệp

Mã sinh viên: B23DCCN292

MỤC LỤC

LỜI CẢM ƠN 7

Chương 1: TỔNG QUAN ĐỀ TÀI 8

1.1. Tên đề tài 8

1.2. Bối cảnh và xu hướng phát triển của IoT 8

1.3. Tính cấp thiết của đề tài 8

1.4. Bài toán đặt ra 9

1.5. Mục tiêu 10

1.6. Đối tượng và phạm vi 11

1.7. Kiến trúc tổng quan hệ thống 11

Chương 2: THIẾT KẾ HỆ THỐNG VÀ GIAO DIỆN 13

I. Phần cứng và phần mềm 13

1. Phần cứng 13

1.1. Vi điều khiển ESP32 (NodeMCU-32S) 13

1.2. Cảm biến nhiệt độ, độ ẩm DHT11 14

1.3. Module quang trở LDR 14

1.4. Khối chấp hành — 3 đèn LED 14

1.5. Sơ đồ đấu nối chân 15

1.6. Mốc thời gian của dữ liệu (NTP) 15

2. Phần mềm 15

II. Thiết kế giao diện 16

2.1. Màn hình Login 16

2.2. Màn hình Dashboard 17

2.3. Màn hình Data Sensor 18

2.4. Màn hình Action History 19

2.5. Màn hình Profile 20

2.6. Điều hướng chung 20

III. Kiến trúc phân lớp và nguyên tắc thiết kế hệ thống 21

1. Kiến trúc phân lớp hệ thống 21

2. Luồng dữ liệu và giao tiếp 21

3. MQTT topic sử dụng 22

4. Nguyên tắc thiết kế và khả năng mở rộng 24

IV. Thiết kế kiến trúc phần mềm Backend (Nuxt.js) 24

V. Thiết kế kiến trúc phần mềm Frontend (Vue.js) 25

Chương 3: USE CASE, SEQUENCE DIAGRAM VÀ API 25

3.1. UC01 – Đăng nhập hệ thống 26

3.2. UC02 – Xem dữ liệu cảm biến thời gian thực 30

3.3. UC03 – Xem biểu đồ lịch sử cảm biến 33

3.4. UC04 – Bật/tắt đèn LED 35

3.5. UC05 – Tra cứu lịch sử cảm biến 39

3.6. UC06 – Tra cứu lịch sử thao tác đèn 41

3.7. Tổng hợp tài liệu API (API Documentation) 44

Chương 4: THIẾT KẾ CƠ SỞ DỮ LIỆU 46

4.1. Mô hình dữ liệu 46

4.2. Bảng Users 48

4.3. Bảng Sensors 48

4.4. Bảng Devices 49

4.5. Bảng SensorData 49

4.6. Bảng ActionHistory 50
4.7. Quy tắc dữ liệu quan trọng 50
Chương 5: KẾT QUẢ VÀ ĐÁNH GIÁ THỰC NGHIỆM 51
5.1. Cấu trúc thư mục dự án 51
5.2. Kết quả triển khai phần cứng 51
5.3. Kết quả triển khai Broker và Backend 52
5.4. Kết quả xây dựng giao diện Frontend 53
5.5. Đánh giá thử nghiệm và kiểm thử hệ thống 59
5.6. Triển khai hệ thống 61
Chương 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 61
6.1. Các hạn chế còn tồn tại 62
6.2. Hướng phát triển 62

DANH MỤC HÌNH ẢNH

Hình 1. Kiến trúc tổng quan hệ thống IoT 12
Hình 2.1: Màn hình đăng nhập 16
Hình 2.2: Màn hình dashboard 17
Hình 2.3: Màn hình hiển thị dữ liệu 18
Hình 2.4: Màn hình lịch sử hoạt động 19
Hình 2.5: Màn hình thông tin cá nhân 20
Hình 3.0: Biểu đồ Use Case tổng quan 26
Hình 3.1: Luồng hoạt động khi đăng nhập 28
Hình 3.2: Luồng hoạt động khi thu thập dữ liệu cảm biến 31
Hình 3.3: Luồng hoạt động khi hiển thị biểu đồ lịch sử 34
Hình 3.4: Luồng hoạt động khi điều khiển trạng thái của đèn led 37
Hình 3.5: Luồng hoạt động khi tra cứu lịch sử cảm biến 40
Hình 3.6: Luồng hoạt động khi tra cứu lịch sử thao tác đèn 43
Hình 4.1: Mô hình cơ sở dữ liệu 47
Hình 5.2: Màn hình đăng nhập (chạy thực tế) 53
Hình 5.3: Màn hình Dashboard 54
Hình 5.4: Đèn LED 3 ở trạng thái LOADING chờ xác nhận 55
Hình 5.5: Đèn LED 3 chuyển sang ON sau khi ESP32 xác nhận 55
Hình 5.6: Hộp thoại ngoại lệ khi lệnh quá thời gian chờ 56
Hình 5.7: Màn hình Data Sensor 57
Hình 5.8: Màn hình Action History 58
Hình 5.9: Màn hình Profile 58

LỜI CẢM ƠN

Lời đầu tiên, em xin bày tỏ lòng biết ơn chân thành tới các thầy cô Học viện Công nghệ Bưu chính Viễn thông, đặc biệt là các thầy cô Khoa Công nghệ Thông tin I, những người đã tận tâm giảng dạy và truyền đạt cho em những kiến thức nền tảng trong suốt quá trình học tập.

Em xin gửi lời cảm ơn sâu sắc tới Thầy Nguyễn Quốc Uy, giảng viên học phần IoT và Ứng dụng, người đã hướng dẫn và góp ý cho em trong suốt quá trình phân tích, thiết kế và hiện thực hệ thống. Những định hướng của Thầy giúp em hiểu rõ hơn cách kết nối phần cứng nhúng, giao thức MQTT và ứng dụng Web thành một hệ thống IoT hoàn chỉnh.

Do kiến thức và kinh nghiệm thực tế còn hạn chế, báo cáo chắc chắn không tránh khỏi thiếu sót. Em rất mong nhận được nhận xét, góp ý của Thầy để tiếp tục hoàn thiện sản phẩm và nâng cao năng lực của bản thân.

Chương 1: TỔNG QUAN ĐỀ TÀI

1.1. Tên đề tài

Hệ thống giám sát nhiệt độ, độ ẩm và ánh sáng môi trường phòng học sử dụng ESP32.

1.2. Bối cảnh và xu hướng phát triển của IoT

Internet of Things (IoT) là xu hướng kết nối các thiết bị vật lý như cảm biến, vi điều khiển, thiết bị chấp hành với Internet để thu thập dữ liệu, giám sát và điều khiển từ xa. Một hệ thống IoT điển hình kết hợp phần cứng nhúng, mạng không dây, giao thức truyền thông nhẹ (như MQTT), máy chủ xử lý và giao diện hiển thị cho người dùng.

IoT đang được ứng dụng rộng rãi trong nhà thông minh, nông nghiệp công nghệ cao, giám sát công nghiệp và đặc biệt là quản lý tòa nhà, phòng học. Điểm chung của các ứng dụng này là biến các đại lượng vật lý (nhiệt độ, độ ẩm, ánh sáng...) thành dữ liệu số, truyền về máy chủ theo thời gian thực và hiển thị trực quan để người quản lý ra quyết định kịp thời.

1.3. Tính cấp thiết của đề tài

Với khí hậu nhiệt đới gió mùa, nhiệt độ và độ ẩm trong phòng học tại Việt Nam thay đổi mạnh theo mùa và theo từng thời điểm trong ngày; ánh sáng tự nhiên cũng không ổn định. Các yếu tố này ảnh hưởng trực tiếp đến sức khỏe, sự tập trung của người học và chi phí điện năng cho chiếu sáng, làm mát.

Cách theo dõi truyền thống bằng nhiệt kế, ẩm kế rời và ghi chép thủ công có nhiều nhược điểm:

- Tốn nhân lực, số liệu có độ trễ và dễ sai sót.
- Không lưu được lịch sử liên tục để phân tích xu hướng hoặc truy vết thời điểm bất thường.
- Không điều khiển được thiết bị (đèn chiếu sáng) từ xa và không biết chắc thiết bị đã thực hiện lệnh hay chưa.

Vì vậy cần một hệ thống IoT khép kín: tự động thu thập dữ liệu từ phần cứng, truyền tin cậy về máy chủ, lưu trữ lịch sử và hiển thị, điều khiển trên giao diện Web theo thời gian thực.

1.4. Bài toán đặt ra

Môi trường phòng học chịu tác động trực tiếp từ ba yếu tố: nhiệt độ, độ ẩm không khí và cường độ ánh sáng — những thông số này thay đổi liên tục trong ngày và ảnh hưởng đến chất lượng giảng dạy, học tập. Đề tài đặt ra yêu cầu xây dựng một công cụ cho phép người quản lý theo dõi các chỉ số này ngay tại thời điểm hiện tại, đồng thời lưu trữ toàn bộ lịch sử đo để có thể đối chiếu, phân tích xu hướng hoặc truy vết lại những thời điểm dữ liệu bất thường đã xảy ra trong quá khứ. Ngoài chức năng quan sát, hệ thống còn phải cho phép thao tác điều khiển đèn chiếu sáng ngay trên giao diện web, và người dùng cần nắm được chính xác thiết bị đã phản hồi lệnh hay chưa.

Một nguyên tắc thiết kế xuyên suốt của hệ thống là tách bạch giữa "yêu cầu điều khiển" mà người dùng gửi đi và "trạng thái vận hành thực tế" của đèn LED. Ngay khi lệnh bật/tắt được phát ra, hệ thống gán trạng thái tạm thời "loading" cho bản ghi thao tác; chỉ khi ESP32 gửi tín hiệu xác nhận, trạng thái mới được chốt lại thành "on" hoặc "off". Thiết kế này ngăn ngừa tình huống giao diện báo sai lệch so với thực tế phần cứng, đặc biệt khi đường truyền chậm hoặc thiết bị mất kết nối tạm thời.

Người dùng có thể truy cập giao diện Web để:

- Xem nhiệt độ hiện tại.
- Xem độ ẩm hiện tại.
- Xem mức ánh sáng hiện tại.
- Xem thời điểm cập nhật gần nhất.
- Xem lịch sử dữ liệu.
- Xem biểu đồ biến động của các thông số.
- Tìm kiếm dữ liệu theo khoảng thời gian.
- Lọc dữ liệu theo loại cảm biến.

Điểm quan trọng của hệ thống là dữ liệu được **lưu lại theo thời gian**, cho phép người dùng không chỉ xem giá trị hiện tại mà còn có thể phân tích sự thay đổi của môi trường trong quá khứ

1.5. Mục tiêu

Hệ thống được xây dựng nhằm đạt các mục tiêu sau:

- Xây dựng hệ thống IoT sử dụng ESP32.
- Đo nhiệt độ bằng cảm biến DHT11.
- Đo độ ẩm bằng cảm biến DHT11.
- Đo mức ánh sáng bằng quang trở LDR.
- Truyền dữ liệu cảm biến tới Backend.
- Lưu dữ liệu cảm biến vào Database.
- Xây dựng RESTful API phục vụ hệ thống.
- Xây dựng giao diện Web Dashboard.
- Hiển thị dữ liệu cảm biến hiện tại.
- Hiển thị biểu đồ lịch sử.
- Cho phép tìm kiếm và lọc dữ liệu.
- Xử lý trường hợp mất kết nối giữa ESP32 và Backend.

Cụ thể theo từng thành phần:

- **Phần cứng:** ESP32 (bo NodeMCU-32S) đọc DHT11 (GPIO 4) và LDR (GPIO 34, quy đổi sang lux) mỗi 2 giây, gắn thời gian thực đồng bộ qua NTP; điều khiển 3 đèn LED ở GPIO 5, 18, 19; tự kết nối lại MQTT khi mất kết nối.
- **Truyền thông:** dùng MQTT qua Broker Mosquitto (cổng 8899, bắt buộc xác thực username/password) với 3 topic iot/sensors/data, iot/devices/control, iot/devices/status; mọi lệnh điều khiển đều có phản hồi trạng thái thật từ phần cứng.

- **Backend:** Nuxt 4 (Nitro) cung cấp RESTful API, xác thực JWT, kênh WebSocket /realtime, lưu dữ liệu vào MySQL và đánh dấu lệnh thất bại (fail) khi quá thời gian chờ.
- **Frontend:** Vue 3 + Vite hiển thị Dashboard 3 thông số và biểu đồ 24 giờ, điều khiển đèn với trạng thái LOADING/ON/OFF/FAIL, tra cứu lịch sử có lọc và phân trang.

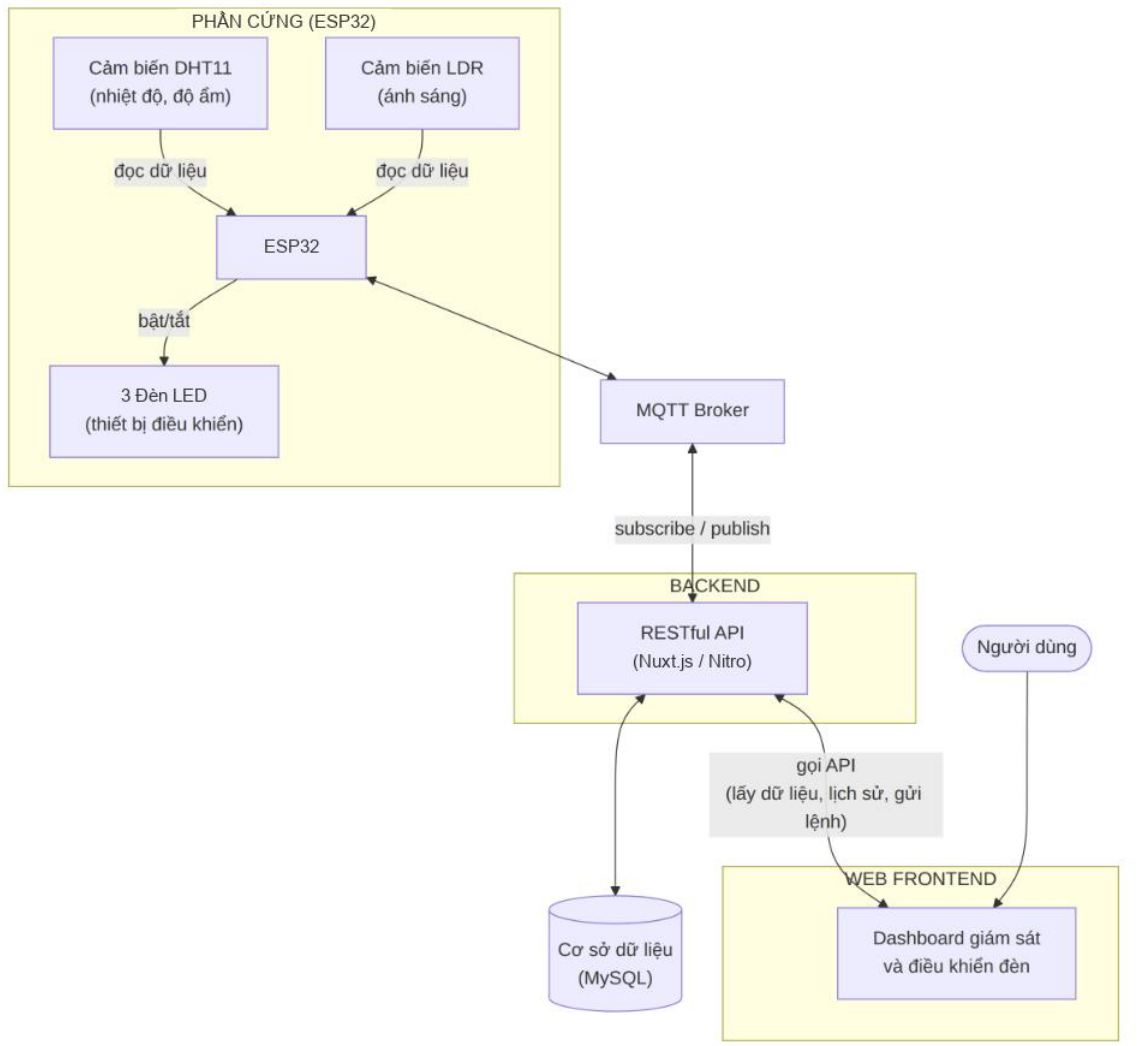
1.6. Đối tượng và phạm vi

Đối tượng nghiên cứu gồm: vi điều khiển ESP32 và cách đọc cảm biến số (DHT11, giao tiếp 1 dây) và cảm biến tương tự (LDR qua bộ ADC); giao thức MQTT với mô hình Publish/Subscribe; nền tảng Nuxt/Nitro cho Backend, Vue 3 cho Frontend và hệ quản trị CSDL MySQL. Báo cáo bao gồm thiết kế giao diện, đặc tả Use Case, Sequence Diagram, thiết kế cơ sở dữ liệu, tài liệu API và kết quả triển khai, kiểm thử thực tế.

Phạm vi hệ thống giới hạn trong một phòng học, quản lý ba đèn LED (Đèn LED 1, 2, 3) và ba loại cảm biến (nhiệt độ, độ ẩm, ánh sáng). Hệ thống hoạt động trong mạng LAN qua Wi-Fi 2.4 GHz. Đối tượng sử dụng là người quản lý phòng học (giảng viên hoặc nhân viên quản trị cơ sở vật chất), truy cập bằng tài khoản đã được cấp qua giao diện Web; hệ thống hiện chưa hỗ trợ phân quyền theo nhiều vai trò hoặc quản lý đồng thời nhiều phòng học.

1.7. Kiến trúc tổng quan hệ thống

Hệ thống được tổ chức theo kiến trúc nhiều lớp. ESP32 đọc dữ liệu từ cảm biến DHT11 và LDR theo chu kỳ 2 giây, sau đó gửi dữ liệu lên MQTT Broker Mosquitto qua topic iot/sensors/data. Backend Nuxt.js nhận dữ liệu, lưu vào cơ sở dữ liệu MySQL, đồng thời cung cấp RESTful API và kênh WebSocket cập nhật thời gian thực cho Frontend Vue.js. Web Frontend là nơi người dùng quan sát dữ liệu, điều khiển đèn và tra cứu lịch sử.



Hình 1. Kiến trúc tổng quan hệ thống IoT

Hệ thống vận hành theo hai luồng chính:

- **Luồng giám sát:** cứ 2 giây ESP32 đọc DHT11 và LDR, gắn thời gian NTP rồi publish JSON lên iot/sensors/data. Backend subscribe topic này, kiểm tra giá trị hợp lệ, lấy thời điểm đo theo timestamp của ESP32 (nếu hợp lệ và lệch không quá 5 phút), ghi vào bảng SensorData rồi đẩy sự kiện sensor:update qua WebSocket để Dashboard cập nhật số liệu và biểu đồ không cần tải lại trang.
- **Luồng điều khiển có xác nhận:** người dùng gạt công tắc, Frontend gọi POST /api/devices/{id}/actions. Backend ghi ActionHistory với status = loading rồi publish lệnh văn bản (vd. LED1_ON) lên iot/devices/control. ESP32 bật/tắt chân GPIO và publish trạng thái thật {"device":"LED1","status":"ON"} lên iot/devices/status.

Backend so khớp: trùng ý định thì chốt on/off kèm confirmed_at, ngược ý định thì đánh dấu fail; quá 15 giây không có phản hồi cũng đánh dấu fail. Sau đó đẩy sự kiện device:status về giao diện.

Chương 2: THIẾT KẾ HỆ THỐNG VÀ GIAO DIỆN

I. Phần cứng và phần mềm

1. Phần cứng

Hệ thống sử dụng các thành phần phần cứng sau:

- Vi điều khiển ESP32 (bo NodeMCU-32S, module ESP-WROOM-32) — đóng vai trò trung tâm xử lý tại chỗ, tích hợp sẵn Wi-Fi để kết nối mạng, đọc dữ liệu từ cảm biến và điều khiển thiết bị chấp hành.
- Cảm biến DHT11 — đo nhiệt độ và độ ẩm không khí, giao tiếp với ESP32 qua 1 chân dữ liệu số (digital pin).
- Quang trở LDR (kèm mạch chia áp) — đo cường độ ánh sáng môi trường, đọc giá trị analog qua 1 chân của ESP32.
- 3 đèn LED (Đèn LED 1, 2, 3) kèm điện trở hạn dòng — thiết bị chấp hành (actuator) mà hệ thống điều khiển bật/tắt.
- Nguồn cấp 5V/3.3V (adapter hoặc cổng USB) — cấp điện cho ESP32 và các module cảm biến.
- Dây nối, breadboard/mạch thử nghiệm — dùng trong giai đoạn lắp ráp và kiểm thử phần cứng.

1.1. Vi điều khiển ESP32 (NodeMCU-32S)

Bo NodeMCU-32S dùng module ESP-WROOM-32 với chip ESP32 của Espressif: bộ xử lý Tensilica Xtensa LX6 32-bit lõi kép, xung nhịp tới 240 MHz, 520 KB SRAM, Flash 4 MB, tích hợp Wi-Fi 802.11 b/g/n (2.4 GHz) và Bluetooth 4.2 BR/EDR/BLE. Chip hoạt động ở 3.3 V; board được cấp nguồn 5 V qua cổng micro-USB và IC ổn áp AMS1117-3.3.

- Bộ ADC 12-bit cho giá trị 0 – 4095; chân GPIO 34 (ADC1_CHANNEL_6, chỉ làm đầu vào) dùng để đọc quang trở LDR.
- Chân GPIO 4 đọc DHT11; GPIO 5, 18, 19 điều khiển 3 đèn LED.
- Firmware dùng vòng lặp không chặn (non-blocking) bằng millis(): client.loop() được gọi liên tục để không bỏ lỡ lệnh điều khiển, cảm biến chỉ được đọc khi đủ 2 giây. So

với ESP32, ESP32 có nhiều GPIO và kênh ADC hơn, SRAM lớn hơn và hai lõi xử lý nên vẫn ổn định khi vừa giữ kết nối Wi-Fi/MQTT vừa đọc cảm biến liên tục.

1.2. Cảm biến nhiệt độ, độ ẩm DHT11

- Dải đo nhiệt độ 0 – 50 °C (sai số ±2 °C); dải đo độ ẩm 20 – 90 %RH (sai số ±5 %RH).
- Giao tiếp một dây (single-bus): mỗi lần đọc trả về 40 bit gồm 16 bit độ ẩm, 16 bit nhiệt độ và 8 bit checksum.
- Nếu đọc lỗi (giá trị NaN), firmware bỏ hai trường temperature, humidity khỏi bản tin của chu kỳ đó và in thông báo lỗi ra Serial; các cảm biến còn lại vẫn được gửi bình thường.

1.3. Module quang trở LDR

Quang trở GL5528 mắc nối tiếp điện trở cố định 10 kΩ trên module thành mạch chia áp. ESP32 đọc điện áp ở GPIO 34 bằng analogRead() (0 – 4095). Với cách mắc của module, giá trị ADC tăng khi trời tối nên firmware đảo giá trị (4095 − giá trị đọc) để số lớn ứng với ánh sáng mạnh, rồi quy đổi sang lux theo đặc tuyến của quang trở: R_LDR = 10 kΩ × (4095 − light) / light và lux = 10 × (R10 / R_LDR)^(1/γ), với R10 = 15 kΩ (điện trở LDR ở 10 lux) và γ = 0.7. Giá trị lux (số nguyên) được gửi lên trường light.

1.4. Khối chấp hành — 3 đèn LED

Ba đèn LED đại diện cho ba khu vực chiếu sáng trong phòng học, được firmware gọi là LED1, LED2, LED3 (GPIO 5, 18, 19) và tương ứng với thiết bị có id 1, 2, 3 trong bảng Devices. Mỗi LED mắc nối tiếp điện trở hạn dòng 220 Ω về GND để bảo vệ chân vi điều khiển. Khi khởi động, các chân được đặt ở mức LOW nên cả 3 đèn đều tắt.

1.5. Sơ đồ đấu nối chân

Linh kiện	Chân linh kiện	Chân ESP32	Chế độ	Chức năng
DHT11	VCC / GND	3V3 / GND	Nguồn	Cấp nguồn cho cảm biến
DHT11	DATA	GPIO 4	INPUT	Đọc nhiệt độ, độ ẩm (1 dây)
Module LDR	VCC / GND	3V3 / GND	Nguồn	Cấp nguồn mạch chia áp
Module LDR	AO	GPIO 34 (ADC1_CH6)	ANALOG	Đọc ánh sáng 0 – 4095, quy đổi lux
Đèn LED 1	Anode (+)	GPIO 5	OUTPUT	LED1: HIGH = bật, LOW = tắt
Đèn LED 2	Anode (+)	GPIO 18	OUTPUT	LED2: HIGH = bật, LOW = tắt
Đèn LED 3	Anode (+)	GPIO 19	OUTPUT	LED3: HIGH = bật, LOW = tắt
3 đèn LED	Cathode (−)	GND qua 220 Ω	—	Hạn dòng, nối mass chung

1.6. Mốc thời gian của dữ liệu (NTP)

ESP32 không có đồng hồ thời gian thực dùng pin nên sau khi kết nối Wi-Fi, firmware gọi configTime() với máy chủ pool.ntp.org, múi giờ GMT+7 (25 200 giây), và chờ getLocalTime() thành công rồi mới bắt đầu làm việc. Mỗi bản tin iot/sensors/data và iot/devices/status đều kèm trường timestamp dạng "YYYY-MM-DD HH:MM:SS" ("N/A" nếu chưa có giờ). Backend dùng timestamp này làm measured_at nếu đúng định dạng và lệch không quá 5 phút so với giờ MySQL; nếu là "N/A", sai định dạng hoặc lệch nhiều thì dùng giờ máy chủ. Mọi giá trị của cùng một lần đo có chung một mốc thời gian.

2. Phần mềm

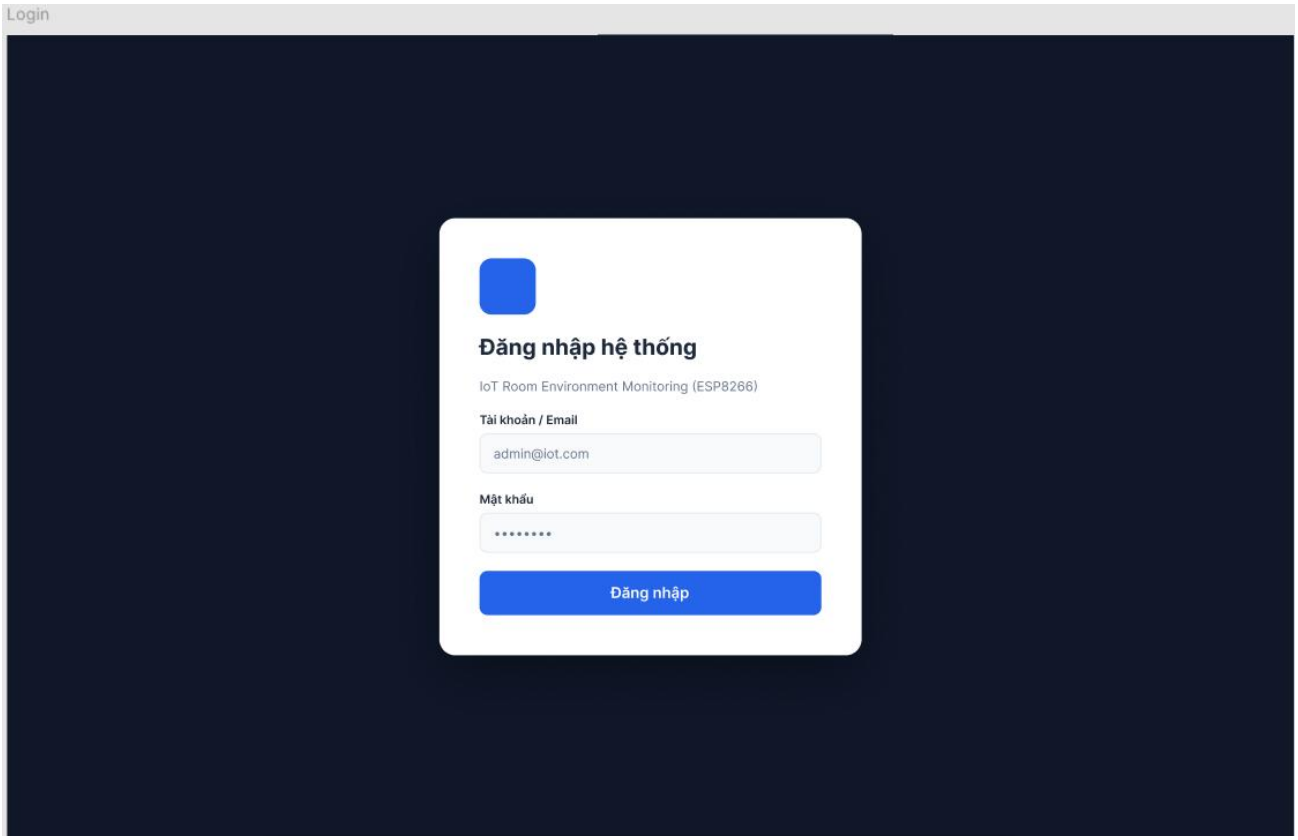
- Firmware ESP32: lập trình bằng Arduino Framework (C/C++) với bộ hỗ trợ Arduino-ESP32, sử dụng các thư viện WiFi.h (kết nối Wi-Fi), PubSubClient (giao tiếp MQTT), DHT sensor library (đọc DHT11) và time.h (đồng bộ NTP). Thông tin Wi-Fi, MQTT đặt trong file secrets.h riêng, không đưa lên GitHub.
- MQTT Broker: Eclipse Mosquitto 2.x tự host trên laptop, cổng 8899, tắt truy cập ẩn danh — trung gian truyền dữ liệu giữa ESP32 và Backend theo mô hình publish/subscribe.
- Backend: Nuxt 4 (chỉ dùng phần server Nitro/h3), thư viện mqtt.js, mysql2, jsonwebtoken, bcryptjs.
- Cơ sở dữ liệu: MySQL (database iot_monitor, 5 bảng).
- Frontend Web: Vue 3 + Vite, Vue Router, Pinia (quản lý trạng thái), Chart.js/vue-chartjs (biểu đồ), Axios (gọi API).
- Công cụ hỗ trợ: Figma (thiết kế UI/UX), Postman (kiểm thử API), Git/GitHub (quản lý mã nguồn và version control)

II. Thiết kế giao diện

Figma "HoangManhHiep-B23DCCN292-IoT" (https://www.figma.com/design/ZMBZDNyG6Ddktflemuibn2) gồm năm màn hình chính: Login, Dashboard, Data Sensor, Action History và Profile. Các màn hình dùng chung sidebar điều hướng nhằm giữ cấu trúc thao tác nhất quán.

2.1. Màn hình Login

Giao diện đăng nhập tối giản với một khối form đặt giữa màn hình, nổi bật trên nền tối. Người dùng nhập tài khoản và mật khẩu, nhấn nút Đăng nhập để hệ thống xác thực; nếu hợp lệ, trình duyệt sẽ tự động điều hướng sang Dashboard.

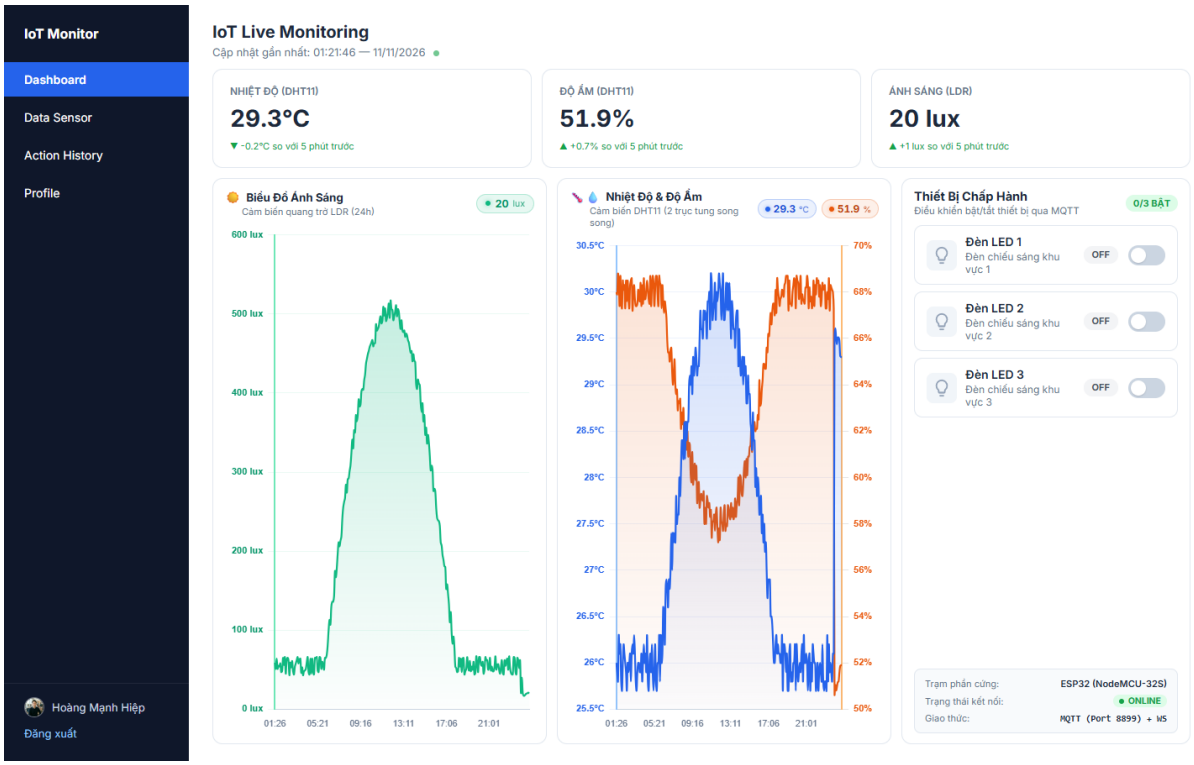


Hình 2.1: Màn hình đăng nhập

2.2. Màn hình Dashboard

Đây là màn hình trung tâm, mang tên "IoT Live Monitoring". Phía trên cùng là ba ô số liệu thể hiện Nhiệt độ, Độ ẩm và Ánh sáng ở thời điểm hiện tại (ví dụ: 28.2°C, 61.9%, 332 lux), có kèm mũi tên thể hiện chiều biến động và mốc thời gian cập nhật gần nhất. Ngay phía dưới là hai biểu đồ 24 giờ: biểu đồ ánh sáng (LDR) và biểu đồ nhiệt độ – độ ẩm (DHT11, hai trục tung). Bên phải là khối "Thiết Bị Chấp Hành" (Actuator Control Panel) gồm ba đèn LED 1, 2, 3, mỗi đèn kèm badge trạng thái LOADING/ON/OFF và công tắc gạt bật/tắt.

Nếu quá lâu không thể kết nối sẽ hiện banner cảnh báo

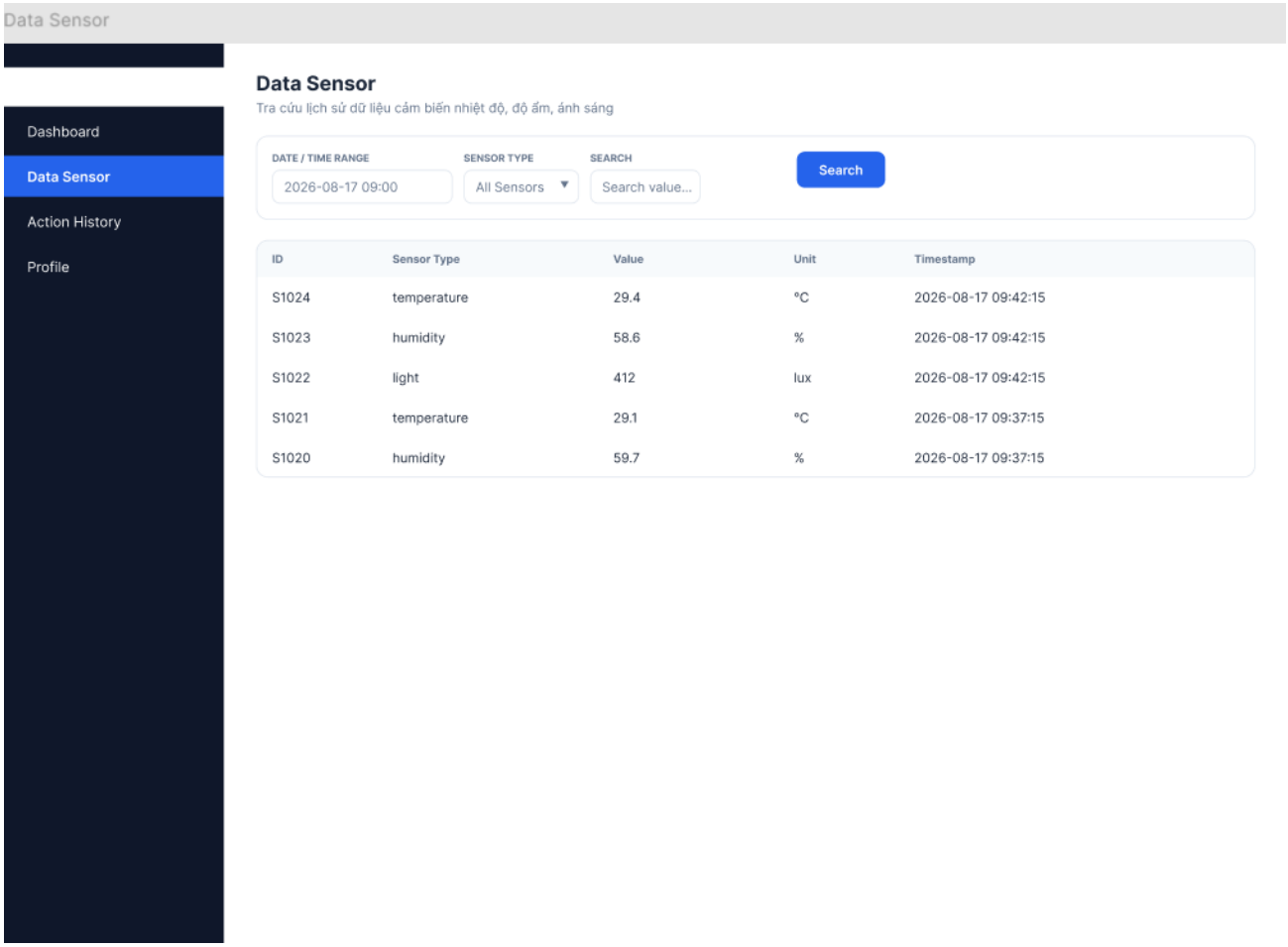


Hình 2.2: Màn hình dashboard

2.3. Màn hình Data Sensor

Màn hình này phục vụ việc tra cứu dữ liệu đo trong quá khứ. Phía trên là thanh lọc gồm khoảng thời gian, loại cảm biến (dạng dropdown list để lựa chọn) và ô tìm kiếm; bên dưới là bảng liệt kê kết quả với các cột ID, loại cảm biến, giá trị, đơn vị đo và thời điểm ghi nhận, có phân trang ở cuối bảng.

Về mặt xử lý, ô tìm kiếm theo thời gian chấp nhận định dạng năm-tháng-ngày giờ:phút:giây, và cho phép nhập thiếu một phần (ví dụ chỉ đến phút) để mở rộng phạm vi tìm kiếm ra cả khung giờ hoặc khung phút tương ứng, ô tìm kiếm có thể tìm kiếm theo ID hoặc đơn vị.

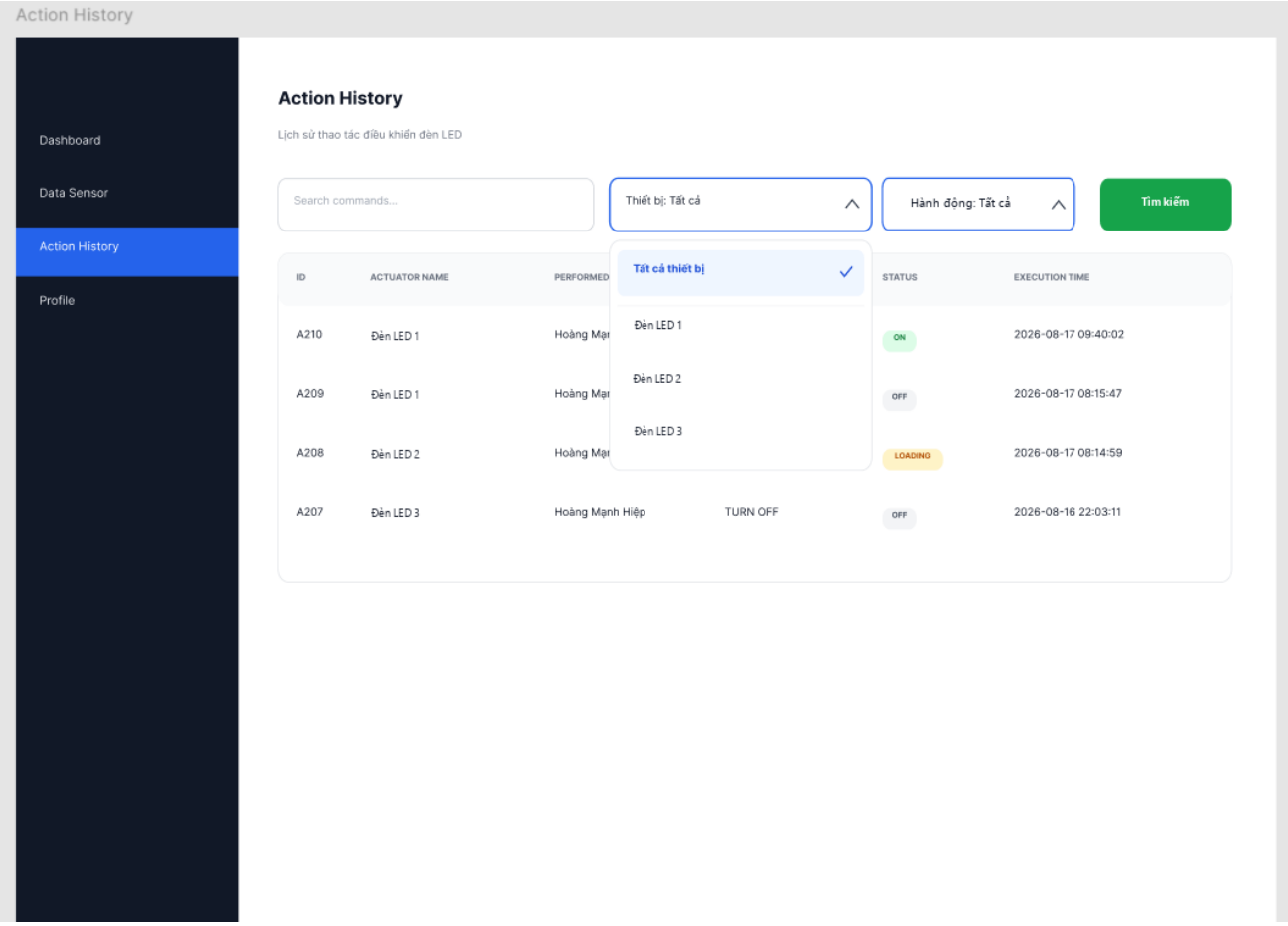


Hình 2.3: Màn hình hiển thị dữ liệu

2.4. Màn hình Action History

Trang này lưu vết mọi thao tác điều khiển đèn LED đã thực hiện. Giao diện gồm ô tìm kiếm theo thời gian, bộ lọc thiết bị / hành động / trạng thái, nút Tìm kiếm và bảng liệt kê với các cột ID, tên thiết bị, hành động đã kích hoạt, người thực hiện kích hoạt, trạng thái (hiển thị dạng badge ON/OFF/LOADING/FAIL) và thời điểm thực thi.

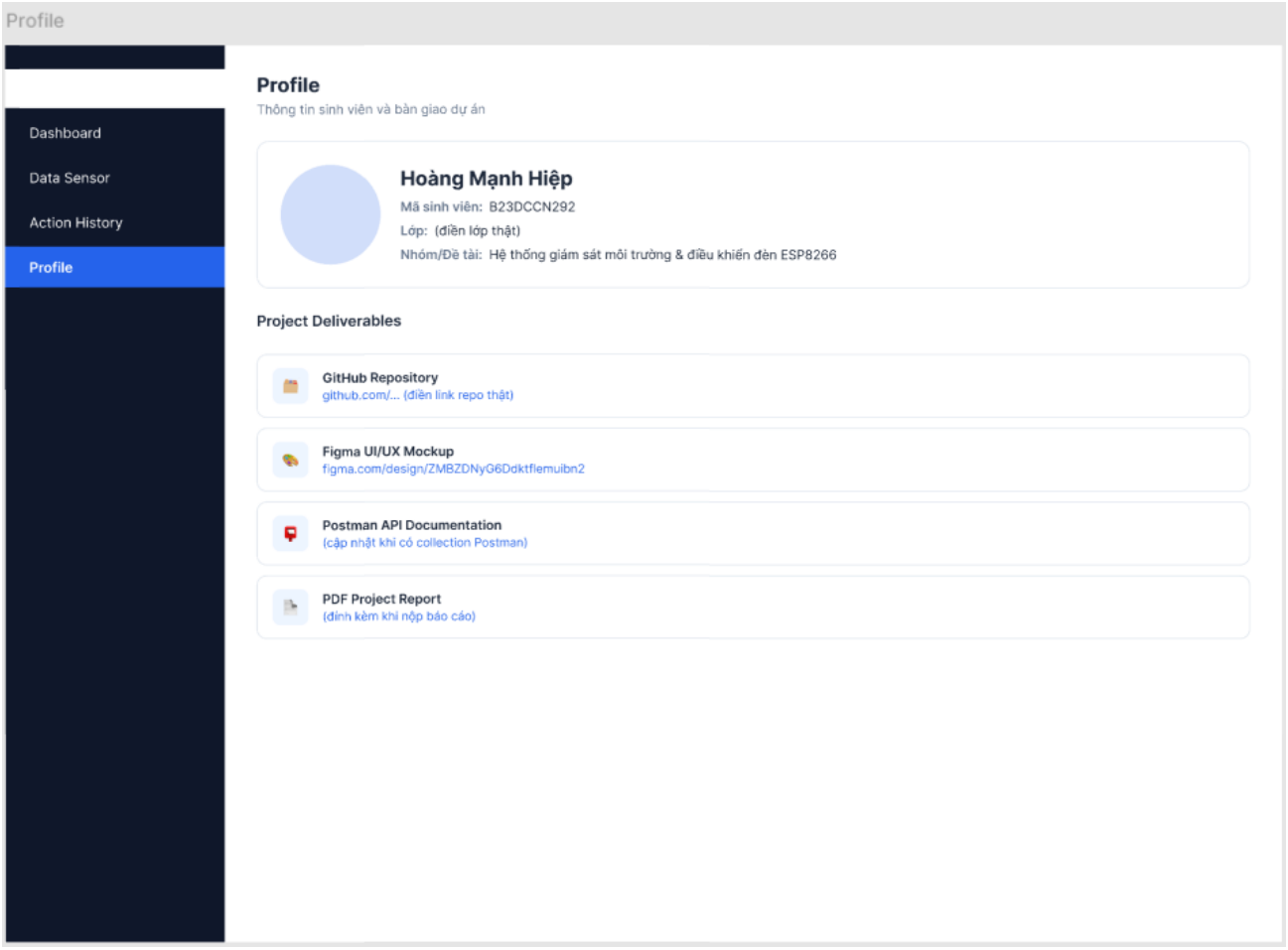
Hệ thống quản lý ba đèn LED (Đèn LED 1, 2, 3), tập hành động gói gọn trong TURN ON và TURN OFF. Trạng thái LOADING xuất hiện trong khoảng thời gian Backend đã ghi nhận yêu cầu nhưng chưa nhận được xác nhận từ ESP32.



Hình 2.4: Màn hình lịch sử hoạt động

2.5. Màn hình Profile

Trang cá nhân hiển thị ảnh đại diện, họ tên, mã số sinh viên và tên đề tài đang thực hiện. Phía dưới là danh sách 4 liên kết bàn giao sản phẩm: GitHub Repository, Figma UI/UX Mockup, Postman API Documentation và PDF Project Report.



Hình 2.5: Màn hình thông tin cá nhân

2.6. Điều hướng chung

Sau đăng nhập, người dùng có thể chuyển giữa Dashboard, Data Sensor, Action History và Profile bằng sidebar. Mục đang hoạt động được tô nền xanh để phân biệt với các mục còn lại.

Ghi chú: Hình 2.2 là ảnh chụp giao diện Dashboard khi chạy thực tế (ba đèn LED 1, 2, 3 nối các chân GPIO 5, 18, 19); các hình còn lại lấy từ bản thiết kế Figma và đã được cập nhật tên thiết bị, bộ lọc cho khớp với hệ thống triển khai.

III. Kiến trúc phân lớp và nguyên tắc thiết kế hệ thống

1. Kiến trúc phân lớp hệ thống

Hệ thống được tổ chức theo các lớp có trách nhiệm tách biệt, giúp việc phát triển, kiểm thử và thay thế từng thành phần không ảnh hưởng đến toàn bộ hệ thống.

Lớp	Thành phần	Trách nhiệm chính
Thiết bị	ESP32 (NodeMCU-32S), DHT11, LDR, 3 đèn LED	Thu thập nhiệt độ, độ ẩm, ánh sáng và thực thi lệnh bật/tắt đèn.
Giao tiếp	MQTT Broker Mosquitto (cổng 8899)	Trung chuyển bản tin cảm biến, lệnh điều khiển và trạng thái theo mô hình Publish/Subscribe.
Ứng dụng	Backend Nuxt.js	Kiểm tra dữ liệu, xử lý nghiệp vụ, cung cấp RESTful API và quản lý kênh WebSocket.
Dữ liệu	MySQL	Lưu thông tin người dùng, cảm biến, thiết bị, dữ liệu đo và lịch sử điều khiển.
Giao diện	Frontend Vue.js	Hiển thị Dashboard, biểu đồ, bảng tra cứu và tiếp nhận thao tác người dùng.

2. Luồng dữ liệu và giao tiếp

Luồng giám sát:

ESP32 đọc DHT11/LDR theo chu kỳ → publish payload lên topic iot/sensors/data → Backend kiểm tra và lưu vào bảng SensorData → Backend phát sự kiện WebSocket sensor:update → Frontend cập nhật ba ô số liệu và biểu đồ trên Dashboard.

Luồng điều khiển:

Khi Dashboard mở, Frontend gọi GET /api/devices để lấy currentStatus mới nhất và hiển thị đúng vị trí công tắc. Khi người dùng bật/tắt đèn, Frontend gửi POST /api/devices/{deviceId}/actions với action ngược trạng thái hiện tại. Backend tạo bản ghi ActionHistory với status=loading, publish lệnh văn bản LEDn_ON / LEDn_OFF lên topic iot/devices/control; ESP32 thực thi, điều khiển GPIO và phản hồi trạng thái thực tế của đèn đó qua topic iot/devices/status; Backend cập nhật status=on/off kèm confirmed_at và đẩy sự kiện WebSocket device:status về Frontend.

Luồng tra cứu:

Frontend gửi request RESTful kèm điều kiện lọc (loại cảm biến, khoảng thời gian, từ khóa...) và phân trang tới Backend → Backend truy vấn SensorData hoặc ActionHistory trong MySQL → trả kết quả kèm thông tin phân trang → Frontend render bảng tại Data Sensor hoặc Action History. Luồng này tách khỏi WebSocket vì dữ liệu lịch sử không cần đẩy liên tục.

3. MQTT topic sử dụng

MQTT (Message Queuing Telemetry Transport) là giao thức nhắn tin nhẹ chạy trên TCP/IP, phù hợp với thiết bị tài nguyên thấp như ESP32. Khác với HTTP theo mô hình Request – Response, MQTT tách rời bên gửi (Publisher) và bên nhận (Subscriber) thông qua một máy chủ trung gian là Broker: thiết bị chỉ cần giữ một kết nối TCP tới Broker, publish dữ liệu lên một topic, và mọi client đã subscribe topic đó sẽ nhận được bản tin.

Cấu hình Broker Mosquitto (file mosquitto.conf, tạo bởi setup.ps1):

```
listener 8899 0.0.0.0
allow_anonymous false
password_file D:/IoT/passwd
```

- Cổng 8899 (không dùng cổng mặc định 1883), mở firewall để ESP32 trong cùng mạng kết nối được.
- Tắt truy cập ẩn danh; tài khoản MQTT được tạo bằng mosquitto_passwd: username HoangManhHiep, password là mã sinh viên B23DCCN292. Backend và ESP32 đều đăng nhập bằng tài khoản này.

Hệ thống sử dụng ba nhóm topic MQTT để tách riêng dữ liệu cảm biến, lệnh điều khiển và phản hồi trạng thái, giúp Backend và ESP32 trao đổi dữ liệu rõ ràng theo mô hình Publish/Subscribe.

Topic	Publisher	Subscriber	Chức năng
iot/sensors/data	ESP32	Backend	Gửi timestamp NTP, nhiệt độ, độ ẩm và ánh sáng (lux); 2 giây/lần.
iot/devices/control	Backend	ESP32	Gửi lệnh văn bản LED1_ON … LED3_OFF, hoặc ALL_ON / ALL_OFF cho cả 3 đèn.
iot/devices/status	ESP32	Backend	Gửi trạng thái thật của từng đèn ngay sau khi thực hiện lệnh (ALL_ON / ALL_OFF sinh 3 bản tin).

Định dạng payload JSON:

Topic iot/sensors/data (ESP32 → Backend):

```
{
  "timestamp": "2026-11-10 09:30:00",
  "temperature": 28.5,
  "humidity": 61.0,
  "light": 320
}
```

Topic iot/devices/control (Backend → ESP32), văn bản thuần, thiết bị id N tương ứng LEDn:

```
LED1_ON | LED1_OFF | LED2_ON | LED2_OFF | LED3_ON | LED3_OFF
ALL_ON | ALL_OFF          // bật/tắt cả 3 đèn
```

Topic iot/devices/status (ESP32 → Backend):

```
{
  "device": "LED1",
```

```
  "status": "ON",
  "timestamp": "2026-11-10 09:30:02"
}
```

Lệnh không đúng định dạng bị ESP32 bỏ qua. Mỗi bản tin trạng thái chỉ nói về một đèn, nên Backend tìm lệnh đang chờ của đúng đèn đó: trạng thái thật trùng ý định của lệnh thì chốt on/off kèm confirmed_at; ngược ý định thì đánh dấu fail và giao diện hiển thị theo trạng thái thật; không có lệnh chờ thì chỉ đồng bộ trạng thái hiện tại của đèn.

4. Nguyên tắc thiết kế và khả năng mở rộng

- Tách trách nhiệm: Frontend chỉ trình bày dữ liệu; Backend xử lý nghiệp vụ và kiểm tra trạng thái; MQTT phụ trách trao đổi với ESP32; MySQL lưu dữ liệu bền vững.
- Trạng thái có xác nhận: thao tác bật/tắt đèn chỉ được coi là hoàn tất khi ESP32 phản hồi qua topic iot/devices/status; quá 15 giây không có phản hồi thì lệnh được đánh dấu fail và đèn trở về trạng thái trước lệnh.
- Mở rộng theo module: có thể bổ sung cảm biến hoặc thiết bị chấp hành mới bằng cách thêm dòng trong bảng danh mục Sensors/Devices và topic MQTT tương ứng, không phải thay đổi luồng xử lý chính.
- An toàn: mật khẩu chỉ lưu dưới dạng hash_password (bcrypt); API được bảo vệ bằng JWT; thông tin kết nối Backend – MQTT – MySQL đặt trong biến môi trường (.env); MQTT bắt buộc xác thực. Khi triển khai thực tế cần bổ sung HTTPS/WSS và MQTT qua TLS.

IV. Thiết kế kiến trúc phần mềm Backend (Nuxt.js)

Backend dùng phần server của Nuxt 4 (Nitro, h3) với cơ chế định tuyến theo file, tổ chức thành các lớp:

- **server/api/** — RESTful API: auth (login, logout), sensors (index, latest, data), devices (index, [id]/actions.post, actions/index, actions/export, actions/[id].patch), users, health.
- **server/plugins/mqtt.ts** — khởi tạo MQTT client khi server chạy, subscribe iot/sensors/data và iot/devices/status; tự kết nối lại sau 3 giây khi mất Broker; khi khởi động, các lệnh còn loading từ phiên trước được chuyển sang fail.
- **server/routes/realtime.ts** — WebSocket /realtime (xác thực bằng token), phát các sự kiện sensor:update, device:status, action:update.

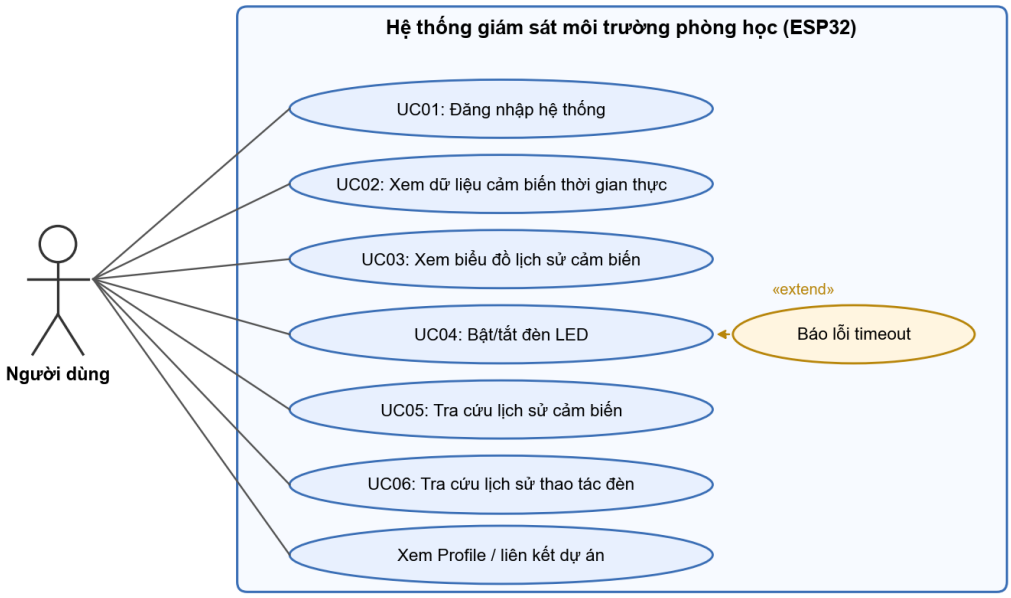
- **server/middleware/** — 01.cors (cho phép origin Frontend), 02.auth (kiểm tra JWT cho mọi API trừ login, logout, health).
- **server/utils/** — db (pool mysql2, câu lệnh tham số hóa), sensors (lưu và đọc dữ liệu cảm biến, kiểm tra ngưỡng hợp lệ), devices (vòng đời lệnh điều khiển, timeout), actions (bộ lọc lịch sử), mqtt, auth, query.
- **database/schema.sql, scripts/db-init.mjs — tạo bảng, seed tài khoản, 3 cảm biến, 3 đèn và dữ liệu mẫu; scripts/esp-simulator.mjs — giả lập ESP32 cùng topic/payload với firmware để kiểm thử khi chưa có phần cứng.**

V. Thiết kế kiến trúc phần mềm Frontend (Vue.js)

- **Công nghệ:** Vue 3 (Composition API) + Vite; Vue Router điều hướng 5 trang, route guard chặn trang khi chưa đăng nhập; Pinia lưu phiên đăng nhập (store auth) và dữ liệu giám sát (store monitor).
- **services/** — http.js (Axios, tự gắn token Bearer), api.js (các hàm gọi API), realtime.js (WebSocket, tự kết nối lại), mock/ (dữ liệu giả lập khi VITE_USE_MOCK=true).
- **views/** — LoginView, DashboardView, DataSensorView, ActionHistoryView, ProfileView.
- **components/** — StatCard (ô số liệu), LightChart, TempHumChart (biểu đồ 24 giờ), DeviceControl, ToggleSwitch (công tắc kèm trạng thái LOADING), OfflineBanner (cảnh báo mất dữ liệu realtime), ExceptionModal (thông báo lệnh FAIL), FilterDropdown, DataPagination.
- **Vite proxy:** /api và /realtime được chuyển tiếp tới Backend http://localhost:3000 khi chạy dev.

Chương 3: USE CASE, SEQUENCE DIAGRAM VÀ API

Toàn hệ thống chỉ có một actor: người dùng đã được cấp quyền truy cập. Sáu Use Case dưới đây bao quát các nghiệp vụ chính mà thiết kế cần đáp ứng.



Hình 3.0: Biểu đồ Use Case tổng quan

Actor	**Mô tả**
Người dùng	Người quản lý phòng học đã được cấp tài khoản (admin@iot.com, user@iot.com); tương tác với hệ thống qua giao diện Web.

Mã	**Use Case**	**Màn hình**
UC01	Đăng nhập hệ thống	Login
UC02	Xem dữ liệu cảm biến thời gian thực	Dashboard
UC03	Xem biểu đồ lịch sử cảm biến	Dashboard
UC04	Bật/tắt thiết bị	Dashboard
UC05	Tra cứu lịch sử cảm biến	Data Sensor
UC06	Tra cứu lịch sử thao tác thiết bị	Action History

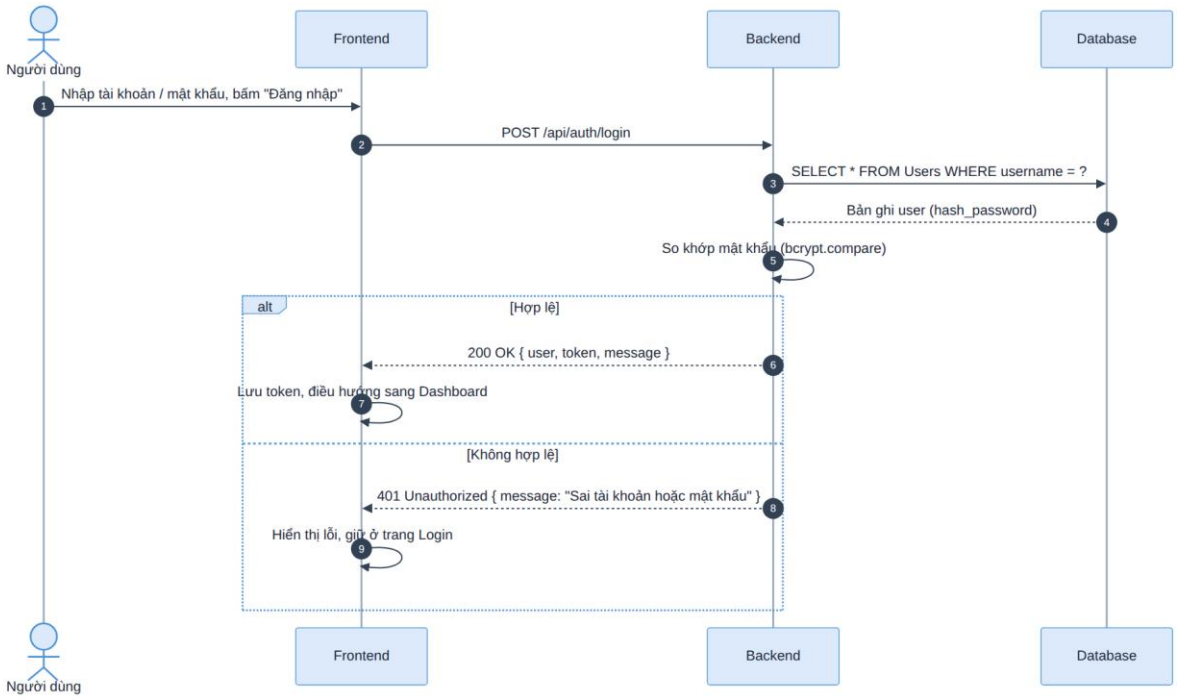
3.1. UC01 – Đăng nhập hệ thống

Thuộc tính	**Mô tả**
Mục tiêu	Kiểm soát quyền truy cập, chỉ cho phép người dùng hợp lệ vào các chức năng

	giám sát/điều khiển.
Actor	Người dùng.
Tiền điều kiện	Tài khoản tồn tại và hợp lệ; hệ thống Backend, Database đang hoạt động bình thường.
Hậu điều kiện	Phiên làm việc được khởi tạo, người dùng được đưa vào Dashboard.
Luồng chính	1) Truy cập trang Login. 2) Nhập tài khoản và mật khẩu. 3) Bấm đăng nhập. 4) Frontend gửi yêu cầu xác thực. 5) Backend kiểm tra thông tin. 6) Nếu hợp lệ, hệ thống trả về phiên đăng nhập và mở Dashboard.
Luồng thay thế	AF-01: Thiếu tài khoản hoặc mật khẩu: Backend trả 400, hiển thị "Vui lòng nhập tài khoản và mật khẩu". AF-02: Sai tài khoản hoặc mật khẩu: Backend trả 401, hiển thị thông báo lỗi và giữ người dùng ở trang Login. AF-03: Token hết hạn / không hợp lệ khi gọi API khác: Backend trả 401, Frontend xóa phiên và chuyển về trang Login.

Sequence Diagram liên quan: Luồng đăng nhập hệ thống

Người dùng nhập tài khoản và mật khẩu trên Frontend, hệ thống gửi yêu cầu xác thực tới Backend. Backend truy vấn bảng Users trong Database, so khớp mật khẩu đã băm; nếu hợp lệ, trả về token đăng nhập kèm thông tin người dùng để Frontend lưu lại và điều hướng sang Dashboard, ngược lại trả về lỗi và giữ người dùng ở trang Login.



Hình 3.1: Luồng hoạt động khi đăng nhập

API liên quan: API đăng nhập

Thuộc tính	**Giá trị**
Method	POST
Endpoint	/api/auth/login
Content-Type	application/json

Request mẫu :

{

"username": "admin@iot.com",

"password": "matkhau123"

}

Response mẫu:

```
{
  "user": {
    "id": 1,
    "fullName": "Hoàng Mạnh Hiệp"
  },
  "message": "Đăng nhập thành công"
}
```

API liên quan: API đăng xuất

Thuộc tính	Giá trị
Method	POST
Endpoint	/api/auth/logout
Content-Type	application/json

Response mẫu:

```
{
  "message": "Đăng xuất thành công"
}
```

Do hệ thống xác thực bằng JWT (stateless), API này chủ yếu để Backend vô hiệu hoá refresh token (nếu có); Frontend luôn xoá token cục bộ ngay khi người dùng bấm đăng xuất.

Mapping dữ liệu API - Database

Trường dữ liệu API	Bảng	Cột
user.id	Users	id
user.fullName	Users	full_name

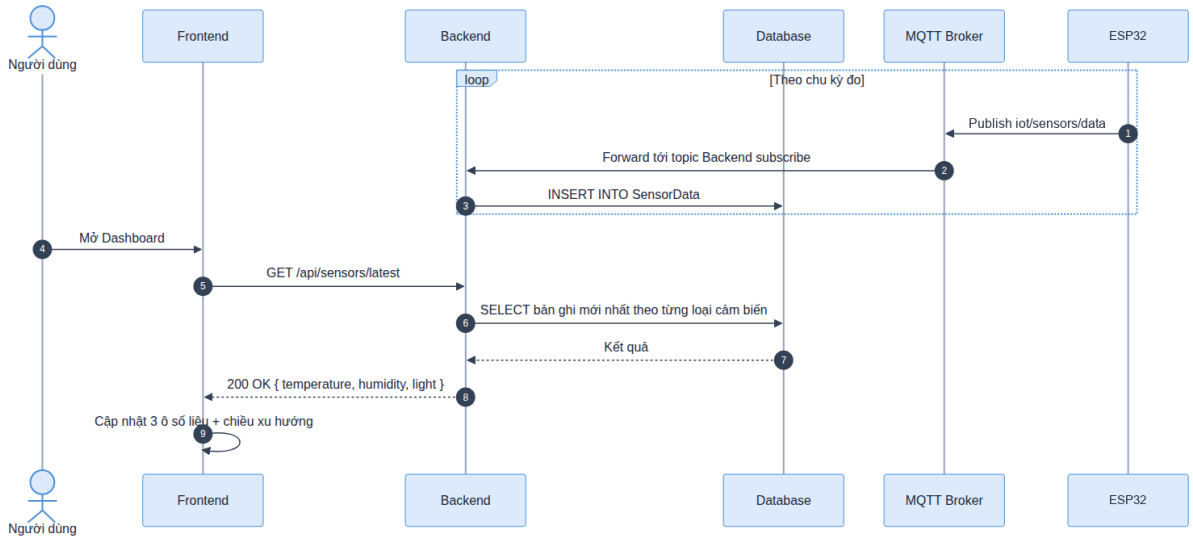
username (request)	Users	username
password (request, đã băm để so khớp)	Users	hash_password

3.2. UC02 – Xem dữ liệu cảm biến thời gian thực

Thuộc tính	Mô tả
Mục tiêu	Cho phép người dùng nắm bắt ngay điều kiện môi trường hiện tại của phòng học.
Actor	Người dùng đã đăng nhập.
Tiền điều kiện	Đang ở màn hình Dashboard; đã có dữ liệu cảm biến gửi về từ ESP32.
Hậu điều kiện	Ba ô hiển thị nhiệt độ/độ ẩm/ánh sáng được cập nhật giá trị và mốc thời gian mới nhất.
Luồng chính	1) Người dùng mở Dashboard.2) Frontend gọi API lấy số liệu mới nhất3) Backend truy vấn bản ghi mới nhất trong SensorData (dữ liệu được ghi từ topic iot/sensors/data).4) Kết quả được hiển thị kèm chiều xu hướng tăng/giảm.
Luồng thay thế	AF-01: ESP32 mất kết nối: giao diện giữ nguyên số liệu cũ; quá 30 giây không có dữ liệu mới thì hiện banner cảnh báo dữ liệu không còn realtime. AF-02: Giá trị cảm biến ngoài ngưỡng hợp lệ (nhiệt độ −40…85 °C, độ ẩm 0…100 %) hoặc không phải số: Backend bỏ qua giá trị đó, không ghi vào CSDL.

Sequence Diagram liên quan: Luồng thu thập dữ liệu cảm biến

ESP32 đọc giá trị từ DHT11 và LDR theo chu kỳ định trước, sau đó publish dữ liệu lên MQTT Broker. Backend lắng nghe topic tương ứng, ghi dữ liệu vào bảng SensorData, đồng thời sẵn sàng phục vụ khi Frontend có nhu cầu hiển thị realtime hoặc tra cứu lịch sử — lúc đó Frontend sẽ gọi API để Backend truy vấn cơ sở dữ liệu và trả kết quả về.



Hình 3.2: Luồng hoạt động khi thu thập dữ liệu cảm biến

API liên quan: API lấy dữ liệu cảm biến mới nhất

Thuộc tính	Giá trị
Method	GET
Endpoint	/api/sensors/latest
MQTT topic	iot/sensors/data (ESP32 publish → Backend subscribe, lưu vào SensorData). API chỉ đọc bản ghi mới nhất trong Database, không gọi MQTT trực tiếp.

Response mẫu:

```
{
  "temperature": {
    "value": 29.4,
    "unit": "°C",
    "updatedAt": "2026-08-17 09:42:15"
  },
  "humidity": {
    "value": 58.6,
    "unit": "%",
    "updatedAt": "2026-08-17 09:42:15"
  },
  "light": {
    "value": 412,
```

```
    "unit": "lux",
    "updatedAt": "2026-08-17 09:42:15"
  }
}
```

API liên quan: Kênh cập nhật thời gian thực (WebSocket)

Thuộc tính	Giá trị
Giao thức	WebSocket
Endpoint	ws://<host>/realtime
Xác thực	Gửi kèm token qua query string hoặc handshake auth
MQTT topic nguồn	sensor:update ← iot/sensors/data device:status ← iot/devices/status

Event mẫu — sensor:update (đẩy khi có dữ liệu cảm biến mới):

```
{
  "event": "sensor:update",
  "data": {
    "temperature": {
      "value": 29.4,
      "unit": "°C",
      "updatedAt": "2026-08-17 09:42:15"
    },
    "humidity": {
      "value": 58.6,
      "unit": "%",
      "updatedAt": "2026-08-17 09:42:15"
    },
    "light": {
      "value": 412,
      "unit": "lux",
      "updatedAt": "2026-08-17 09:42:15"
    }
  }
}
```

Event mẫu — device:status (đẩy khi ESP32 xác nhận trạng thái đèn):

```
{
  "event": "device:status",
```

```
  "data": {
    "deviceId": 1,
    "status": "on",
    "confirmedAt": "2026-08-17 09:45:04"
  }
}
```

Mapping dữ liệu API - Database

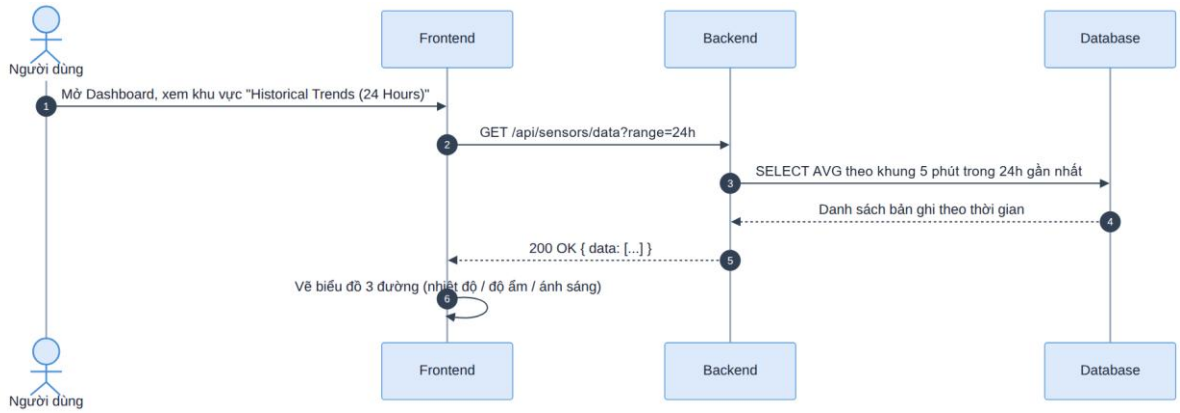
Trường dữ liệu API	Bảng	Cột
temperature/humidity/light .value	SensorData	value
temperature/humidity/light .unit	Sensors	unit
temperature/humidity/light .updatedAt	SensorData	measured_at
data.deviceId (event device:status)	Devices	id
data.status (event device:status)	ActionHistory	status
data.confirmedAt (event device:status)	ActionHistory	confirmed_at

3.3. UC03 – Xem biểu đồ lịch sử cảm biến

Thuộc tính	Mô tả
Mục tiêu	Giúp người dùng quan sát diễn biến các chỉ số môi trường theo thời gian.
Actor	Người dùng đã đăng nhập.
Tiền điều kiện	Bảng SensorData đã có dữ liệu tích lũy.
Hậu điều kiện	Biểu đồ hiển thị đủ 3 chuỗi dữ liệu theo trục thời gian..
Luồng chính	1) Người dùng ở Dashboard 2) Frontend yêu cầu dữ liệu lịch sử 24 giờ. 3) Backend truy vấn SensorData theo điều kiện thời gian. 4) Dữ liệu trả về được Frontend vẽ thành biểu đồ.
Luồng thay thế	AF-01: Không có dữ liệu: hiển thị biểu đồ rỗng hoặc thông báo chưa có dữ liệu. AF-02: Tham số range sai định dạng (vd. 24x): Backend trả 400.

Sequence Diagram liên quan: Luồng hiển thị biểu đồ lịch sử cảm biến

Khi người dùng mở khu vực "Historical Trends (24 Hours)" trên Dashboard, Frontend gọi API lấy dữ liệu lịch sử trong khung 24 giờ gần nhất. Backend truy vấn bảng SensorData theo điều kiện thời gian và trả kết quả về dạng danh sách; Frontend dùng dữ liệu này để vẽ biểu đồ 3 đường tương ứng ba loại cảm biến.



Hình 3.3: Luồng hoạt động khi hiển thị biểu đồ lịch sử

API liên quan: API lấy lịch sử dữ liệu cảm biến

Thuộc tính	**Giá trị**
Method	GET
Endpoint	/api/sensors/data
Query params	type, time, search, page, limit

Request mẫu: GET /api/sensors/data?type=temperature&time=2026-08-17%2009:30&page=1&limit=20

Response mẫu:

```
{
  "data": [
    {
      "id": "S1024",
      "sensorType": "temperature",
      "value": 29.4,
      "unit": "°C",
      "timestamp": "2026-08-17 09:42:15"
    }
  ],
  "page": 1,
  "total": 864
}
```


Mapping dữ liệu API - Database

Trường dữ liệu API	Bảng	Cột
data[].id	SensorData	id
data[].sensorType	Sensors	type
data[].value	SensorData	value
data[].unit	Sensors	unit
data[].timestamp	SensorData	measured_at

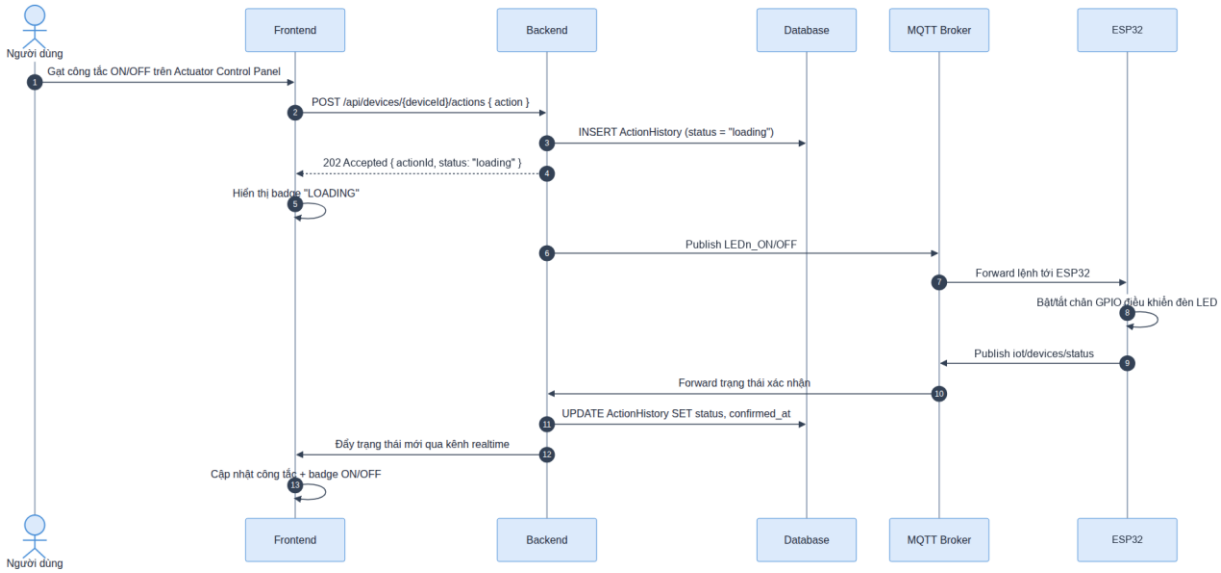
3.4. UC04 – Bật/tắt đèn LED

Thuộc tính	Mô tả
Mục tiêu	Cho phép điều khiển đèn LED từ xa và đảm bảo giao diện phản ánh đúng trạng thái vật lý của đèn.
Actor	Người dùng đã đăng nhập.
Tiền điều kiện	Đèn LED đã được đăng ký trong hệ thống; kết nối MQTT giữa Backend và ESP32 ổn định
Hậu điều kiện	Bản ghi thao tác được lưu vào ActionHistory; trạng thái đèn được cập nhật đúng theo phản hồi phần cứng.
Luồng chính	1) Người dùng thao tác công tắc của một đèn trên Dashboard. 2) Frontend gửi POST /api/devices/{deviceId}/actions với action ON/OFF. 3) Backend khởi tạo bản ghi ActionHistory với status=loading, đổi

	current_status của đèn sang loading. 4) Backend publish lệnh LEDn_ON / LEDn_OFF lên topic iot/devices/control. 5) ESP32 thực thi và điều khiển vật lý đèn LED. 6) ESP32 publish trạng thái thật {"device":"LEDn","status":"ON|OFF"} lên topic iot/devices/status. 7) Backend so khớp với lệnh đang chờ: trùng thì cập nhật status=on/off và confirmed_at, ngược thì status=fail. 8) Giao diện cập nhật lại công tắc và badge tương ứng.
Luồng thay thế	AF-01: Không nhận phản hồi phần cứng: sau 15 giây Backend đánh dấu status = fail, trả công tắc về trạng thái cũ và đẩy sự kiện action:update; Frontend hiện hộp thoại ngoại lệ. AF-02: Đèn đang ở trạng thái loading (lệnh trước chưa xác nhận): Backend trả 409 Conflict. AF-03: Backend mất kết nối MQTT Broker: trả 503, không ghi lệnh mới. AF-04: Thiết bị không tồn tại: trả 404; action khác ON/OFF: trả 400.

Sequence Diagram liên quan: Luồng điều khiển đèn LED

Khi người dùng thao tác công tắc trên Dashboard, Frontend gửi action tới Backend. Thay vì giả định đèn đã đổi trạng thái ngay, Backend lưu bản ghi ActionHistory với status=loading rồi mới publish lệnh qua MQTT. Chỉ khi ESP32 phản hồi trạng thái thật, Backend mới cập nhật status thành on/off (kèm confirmed_at) và đẩy trạng thái mới về giao diện.



Hình 3.4: Luồng hoạt động khi điều khiển trạng thái của đèn led

API liên quan: API lấy danh sách thiết bị

Thuộc tính	Giá trị
Method	GET
Endpoint	/api/devices
MQTT topic	Không gọi MQTT trực tiếp; currentStatus được cập nhật khi Backend nhận bản tin từ iot/devices/status.

Response mẫu:

```
{
  "data": [
    {
      "id": 1,
      "name": "Đèn LED 1",
      "description": "Đèn chiếu sáng khu vực 1",        "currentStatus":
"on"
    }
  ]
}
```

API liên quan: API gửi lệnh điều khiển đèn LED

Thuộc tính	Giá trị
Method	POST
Endpoint	/api/devices/{deviceId}/actions
Content-Type	application/json
MQTT publish	iot/devices/control — Payload: LED1_ON (văn bản thuần)
MQTT subscribe	iot/devices/status — Payload: {"device":"LED1","status":"ON","timestamp":"2026-11-10 09:30:02"}

Request mẫu: { "action": "ON" }

Response mẫu:

```
{
  "actionId": "A211",
  "deviceId": 1,
  "action": "ON",
  "status": "loading",
  "createdAt": "2026-08-17T09:45:02Z",
  "message": "Yêu cầu đã được ghi nhận và chuyển tới MQTT"
}
```

API liên quan: sử dụng lại kênh cập nhật thời gian thực (đã trình bày ở mục 3.2 – UC02), sự kiện "device:status" dùng để đẩy trạng thái on/off sau khi ESP32 xác nhận.

Mapping dữ liệu API - Database

Trường dữ liệu API	Bảng	Cột
data[].id (danh sách thiết bị)	Devices	id
data[].name	Devices	name

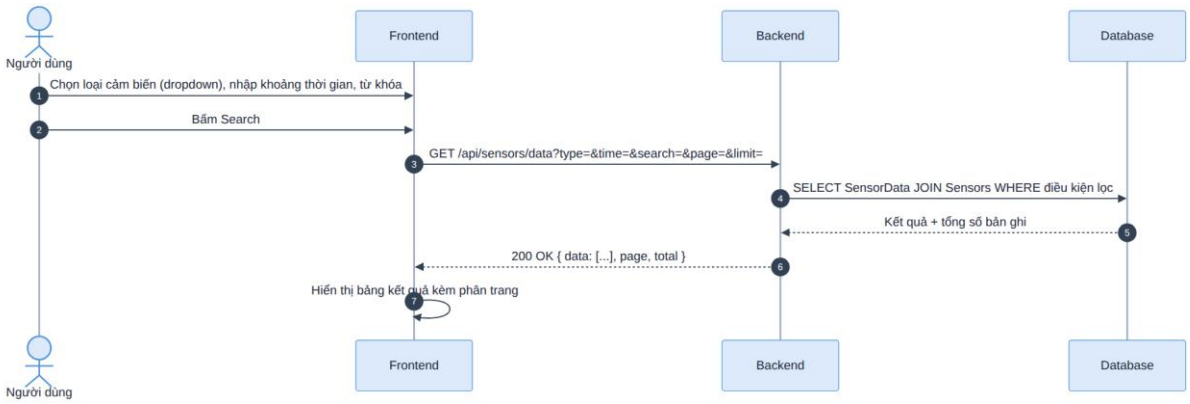
data[].currentStatus	Devices	current_status
actionId	ActionHistory	id
deviceId	ActionHistory	device_id
action	ActionHistory	action
status	ActionHistory	status
createdAt	ActionHistory	created_at

3.5. UC05 – Tra cứu lịch sử cảm biến

Thuộc tính	**Mô tả**
Mục tiêu	Hỗ trợ tìm lại các phép đo theo loại cảm biến (chọn từ dropdown list), khoảng thời gian hoặc từ khóa tìm kiếm.
Actor	Người dùng đã đăng nhập.
Tiền điều kiện	Có dữ liệu trong SensorData.
Hậu điều kiện	Bảng kết quả tại Data Sensor hiển thị đúng dữ liệu thỏa điều kiện lọc, có phân trang.
Luồng chính	1) Vào Data Sensor. 2) Chọn loại cảm biến cần lọc từ dropdown list (hoặc bỏ qua). 3) Nhập khoảng thời gian. 4) Tùy chọn nhập từ khóa vào ô tìm kiếm. 5) Bấm Search. 6) Backend dựng câu truy vấn tương ứng. 7) Kết quả trả về hiển thị đầy đủ ID, loại cảm biến, giá trị, đơn vị, timestamp.
Luồng thay thế	AF-01: Không có kết quả phù hợp: hiển thị danh sách rỗng và thông báo không tìm thấy dữ liệu. AF-02: Thời gian nhập sai định dạng yyyy/mm/dd hh:mm:ss: Backend trả 400.

Sequence Diagram liên quan: Luồng tra cứu lịch sử cảm biến

Tại màn hình Data Sensor, người dùng chọn loại cảm biến từ dropdown, nhập khoảng thời gian và từ khóa tìm kiếm rồi bấm Search. Frontend gửi các tham số lọc này tới Backend; Backend truy vấn bảng SensorData kết hợp bảng Sensors theo điều kiện lọc, trả về kết quả kèm thông tin phân trang để Frontend hiển thị bảng dữ liệu.



Hình 3.5: Luồng hoạt động khi tra cứu lịch sử cảm biến

API liên quan: sử dụng lại API lấy lịch sử dữ liệu cảm biến (đã trình bày ở mục 3.3 – UC03) với các tham số lọc type/time/search tương ứng.

API liên quan: API lấy danh sách loại cảm biến

Thuộc tính	Giá trị
Method	GET
Endpoint	/api/sensors

Response mẫu:

```
{
  "data": [
    { "id": 1, "name": "Nhiệt độ", "type": "temperature", "unit": "°C",
"warnAbove": null },
    { "id": 2, "name": "Độ ẩm",     "type": "humidity",     "unit": "%",
"warnAbove": null },
    { "id": 3, "name": "Ánh sáng", "type": "light",        "unit": "lux",
"warnAbove": null }
  ]
}
```

API này phục vụ dropdown "SENSOR TYPE" ở màn hình Data Sensor, lấy trực tiếp từ bảng danh mục Sensors — đúng theo nguyên tắc mở rộng đã nêu ở mục III.4 (thêm cảm biến mới chỉ cần thêm dòng trong bảng, không cần sửa Frontend).

Mapping dữ liệu API - Database

Trường dữ liệu API	Bảng	Cột
data[].id (danh sách loại cảm biến)	Sensors	id
data[].name	Sensors	name
data[].type	Sensors	type
data[].unit	Sensors	unit

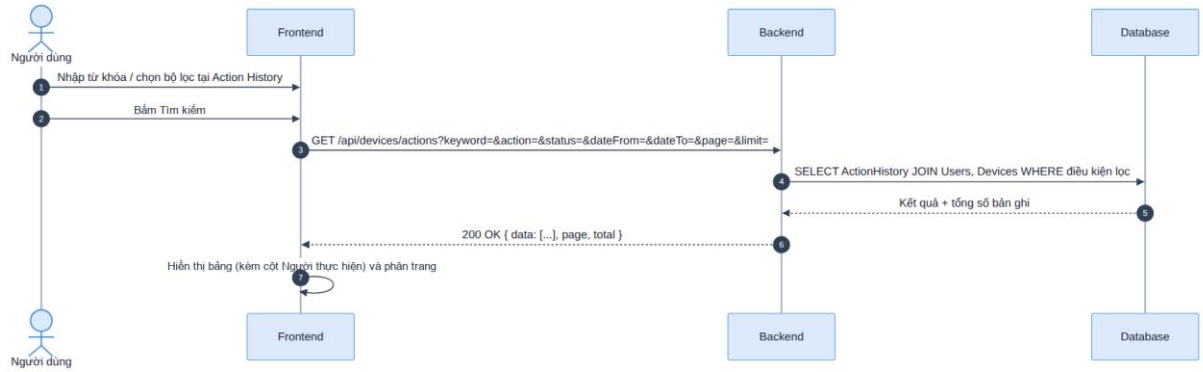
3.6. UC06 – Tra cứu lịch sử thao tác đèn

Thuộc tính	Mô tả
Mục tiêu	Cho phép xem lại các lần điều khiển đèn: hành động nào, kết quả ra sao, thực

	hiện lúc nào.
Actor	Người dùng đã đăng nhập.
Tiền điều kiện	ActionHistory có dữ liệu.
Hậu điều kiện	Bảng Action History hiển thị đúng kết quả tìm kiếm/lọc, người điều khiển, hỗ trợ phân trang.
Luồng chính	1) Vào Action History. 2) Nhập từ khóa hoặc chọn điều kiện lọc (hành động/trạng thái/thời gian). 3) Frontend gọi API tương ứng. 4) Backend truy vấn ActionHistory. 5) Kết quả hiển thị đầy đủ ID, tên thiết bị, hành động, trạng thái, thời gian thực thi.
Luồng thay thế	AF-01: Không có dữ liệu phù hợp: bảng rỗng, không phát sinh lỗi hệ thống. AF-02: Tham số lọc không hợp lệ (action khác ON/OFF, status ngoài loading/on/off/fail, dateFrom sai định dạng): Backend trả 400.

Sequence Diagram liên quan: Luồng tra cứu lịch sử thao tác đèn

Tại màn hình Action History, người dùng nhập từ khóa hoặc chọn bộ lọc (hành động, trạng thái, khoảng thời gian) rồi bấm Tìm kiếm. Frontend gửi yêu cầu tới Backend; Backend truy vấn bảng ActionHistory kết hợp Users và Devices để lấy đầy đủ thông tin người thực hiện, trả kết quả kèm phân trang cho Frontend hiển thị.



Hình 3.6: Luồng hoạt động khi tra cứu lịch sử thao tác đèn

API liên quan: API lấy lịch sử thao tác đèn

Thuộc tính	Giá trị
Method	GET
Endpoint	/api/devices/actions
Query params	keyword, action, status, dateFrom, dateTo, page, limit

Response mẫu:

```
{
  "id": "A211",
  "deviceName": "Đèn LED 1",
  "performedBy": "Hoàng Mạnh Hiệp",
  "action": "ON",
  "status": "on",
  "createdAt": "2026-08-17 09:45:02",
  "confirmedAt": "2026-08-17 09:45:04"
}
```

API liên quan: API xuất CSV lịch sử thao tác

Thuộc tính	Giá trị
Method	GET
Endpoint	/api/devices/actions/export
Query params	keyword, action, status, dateFrom, dateTo
Response	File CSV (Content-Type: text/csv), chứa toàn bộ bản ghi thỏa điều kiện lọc (không phân trang)

API này xuất toàn bộ lịch sử thao tác thỏa điều kiện lọc ra file CSV (gọi trực tiếp kèm token, ví dụ qua Postman); giao diện Action History hiện dùng nút Tìm kiếm, chưa gắn nút xuất CSV.

Mapping dữ liệu API - Database

Trường dữ liệu API	Bảng	Cột
id	ActionHistory	id
deviceName	Devices	name
performedBy	Users	full_name
action	ActionHistory	action
status	ActionHistory	status
createdAt	ActionHistory	created_at
confirmedAt	ActionHistory	confirmed_at

3.7. Tổng hợp tài liệu API (API Documentation)

Base URL: http://localhost:3000/api — **Content-Type:** application/json — **Xác thực:** JWT Bearer Token, gửi qua header Authorization: Bearer <token> (token trả về từ API đăng nhập, hạn 1 ngày). WebSocket truyền token qua query string ?token=.

STT	Method	Endpoint	Mô tả	Quyền
1	POST	/api/auth/login	Đăng nhập, trả token + thông tin người dùng	Public
2	POST	/api/auth/logout	Đăng xuất, thu hồi token	Public
3	GET	/api/sensors/latest	Giá trị mới nhất của từng cảm biến	Đăng nhập
4	GET	/api/sensors	Danh mục loại cảm biến (dropdown)	Đăng nhập
5	GET	/api/sensors/data	Lịch sử cảm biến: type, time, search, page, limit; range=24h cho biểu đồ	Đăng nhập
6	GET	/api/devices	Danh sách 3 đèn kèm currentStatus	Đăng nhập
7	POST	/api/devices/{id}/actions	Gửi lệnh ON/OFF, trả 202 + status loading	Đăng nhập
8	GET	/api/devices/actions	Lịch sử thao tác: keyword, action, status, dateFrom, dateTo, page, limit	Đăng nhập
9	GET	/api/devices/actions/export	Xuất CSV lịch sử thao tác (UTF-8)	Đăng nhập
10	PATCH	/api/devices/actions/{actionId}	Frontend báo lệnh quá thời gian chờ ({"status":"fail"})	Đăng nhập
11	GET	/api/users	Danh sách người dùng (bộ lọc người thực hiện)	Đăng nhập
12	GET	/api/health	Trạng thái kết nối MySQL / MQTT	Public
13	WS	/realtime	Sự kiện sensor:update, device:status, action:update	Token

Mã trạng thái HTTP dùng chung:

Mã	Ý nghĩa
200	Thành công
202	Lệnh điều khiển đã được ghi nhận và gửi tới Broker, chờ ESP32 xác nhận
400	Thiếu hoặc sai tham số (action không phải ON/OFF, thời gian sai định dạng...)
401	Sai tài khoản/mật khẩu, thiếu token hoặc token hết hạn
404	Không tìm thấy thiết bị / bản ghi
409	Thiết bị đang chờ xác nhận lệnh trước đó (loading)
503	Backend chưa kết nối được MQTT Broker

Response lỗi mẫu:

```
{
  "statusCode": 409,
```

```
  "statusMessage": "Conflict",
  "message": "Thiết bị đang chờ xác nhận lệnh trước đó"
}
```

API báo lệnh quá thời gian chờ (PATCH /api/devices/actions/{actionId}) — chỉ đổi được khi bản ghi còn loading; Backend trả công tắc về trạng thái trước lệnh.

Request mẫu: { "status": "fail" }

```
{
  "success": true,
  "action": {
    "id": "A11",
    "deviceName": "Đèn LED 2",
    "performedBy": "Hoàng Mạnh Hiệp",
    "action": "ON",
    "status": "fail",
    "createdAt": "2026-11-03 15:33:30",
    "confirmedAt": null
  }
}
```

API kiểm tra trạng thái (GET /api/health):

```
{
  "status": "ok",
  "database": true,
  "mqtt": true,
  "realtimeClients": 1
}
```

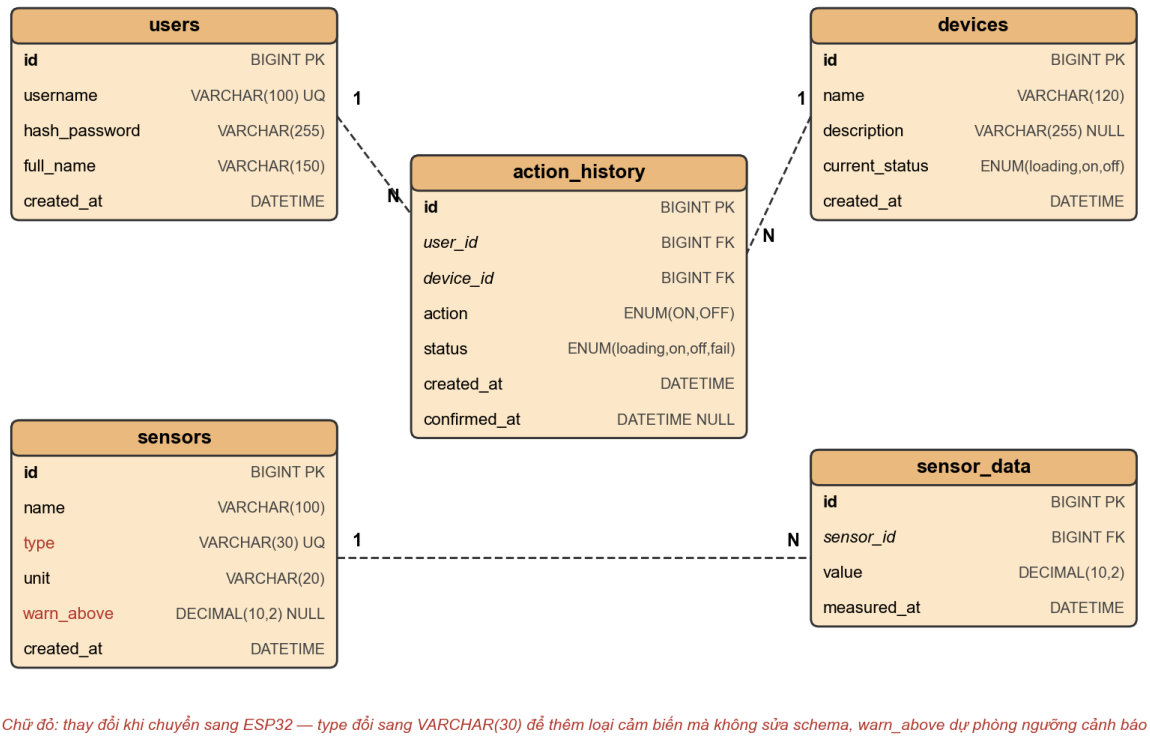
Chương 4: THIẾT KẾ CƠ SỞ DỮ LIỆU

4.1. Mô hình dữ liệu

Lược đồ đề xuất gồm năm bảng: Users, Sensors, Devices, SensorData và ActionHistory. Cách tổ chức này tách riêng phần "danh mục" (Sensors, Devices — mô tả cảm biến/thiết bị nào đang tồn tại trong hệ thống) khỏi phần "dữ liệu phát sinh theo thời gian" (SensorData, ActionHistory — ghi nhận số đo và thao tác điều khiển). Nhờ vậy, nếu sau này bổ sung thêm cảm biến hoặc thiết bị điều khiển mới, chỉ cần thêm dòng vào bảng danh mục mà không phải sửa đổi cấu trúc các bảng lịch sử.

Trong quá trình triển khai, lược đồ được bổ sung hai điểm: bảng Devices có thêm cột description (mô tả hiển thị dưới tên đèn) và current_status (trạng thái hiện tại của đèn), bảng ActionHistory có thêm giá trị status = 'fail' cho cơ chế timeout. Tên bảng vật lý trong MySQL dùng dạng snake_case: users, sensors, devices, sensor_data, action_history.

Khi chuyển sang ESP32, cột type của bảng Sensors đổi từ ENUM sang VARCHAR(30) để có thể thêm loại cảm biến mới chỉ bằng một dòng dữ liệu, không phải sửa cấu trúc bảng; cột warn_above được thêm để dự phòng ngưỡng cảnh báo cho các cảm biến bổ sung sau này (hiện để NULL). Hệ thống hiện có 3 cảm biến: nhiệt độ, độ ẩm, ánh sáng. Lệnh npm run db:init tự nâng cấp database cũ tại chỗ mà không mất dữ liệu.



Hình 4.1: Mô hình cơ sở dữ liệu

4.2. Bảng Users

Trường	Kiểu dữ liệu	Ràng buộc	Nullable	Mô tả
id	BIGINT	PK, AUTO_INCREMENT	NO	Khóa chính người dùng.
username	VARCHAR(100)	UNIQUE	NO	Tên đăng nhập/email.
hash_password	VARCHAR(255)	—	NO	Mật khẩu đã băm bcrypt.
full_name	VARCHAR(150)	—	NO	Họ tên người dùng.
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	NO	Thời điểm tạo tài khoản.

4.3. Bảng Sensors

Trường	Kiểu dữ liệu	Ràng buộc	Nullable	Mô tả
id	BIGINT	PK, AUTO_INCREMENT	NO	Mã cảm biến.
name	VARCHAR(100)	—	NO	Tên cảm biến.
type	VARCHAR(30)	UNIQUE	NO	Loại phép đo, trùng tên trường trong payload iot/sensors/data: temperature, humidity, light.
unit	VARCHAR(20)	—	NO	°C, %, lux.
warn_above	DECIMAL(10,2)	—	YES	Ngưỡng cảnh báo dự phòng cho cảm biến bổ sung; hiện NULL (không cảnh báo).
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	NO	Thời điểm đăng ký cảm biến.

4.4. Bảng Devices

Trường	Kiểu dữ liệu	Ràng buộc	Nullable	Mô tả
id	BIGINT	PK, AUTO_INCREMENT	NO	Mã thiết bị; id N tương ứng tên LEDn trong MQTT.
name	VARCHAR(120)	—	NO	Tên thiết bị, ví dụ "Đèn LED 1".
description	VARCHAR(255)	—	YES	Mô tả ngắn hiển thị dưới tên đèn.
current_status	ENUM('loading','on','off')	DEFAULT 'off'	NO	Trạng thái hiện tại của thiết bị.
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	NO	Thời điểm đăng ký thiết bị.

4.5. Bảng SensorData

Trường	Kiểu dữ liệu	Ràng buộc	Nullable	Mô tả
id	BIGINT	PK, AUTO_INCREMENT	NO	Mã bản ghi.
sensor_id	BIGINT	FK → sensors.id, INDEX, ON DELETE RESTRICT	NO	Cảm biến tạo dữ liệu.
value	DECIMAL(10,2)	—	NO	Giá trị đo.
measured_at	DATETIME	INDEX (sensor_id, measured_at)	NO	Thời điểm đo: timestamp NTP của ESP32, hoặc giờ máy chủ nếu N/A / lệch > 5 phút.

4.6. Bảng ActionHistory

Trường	Kiểu dữ liệu	Ràng buộc	Nullable	Mô tả
id	BIGINT	PK, AUTO_INCREMENT	NO	Mã thao tác (hiển thị dạng A{id}).
user_id	BIGINT	FK → users.id, INDEX, ON DELETE RESTRICT	NO	Người gửi lệnh.
device_id	BIGINT	FK → devices.id, INDEX, ON DELETE RESTRICT	NO	Thiết bị được điều khiển.
action	ENUM('ON','OFF')	—	NO	Ý định điều khiển do người dùng gửi đi.
status	ENUM('loading','on','off','fail')	DEFAULT 'loading'	NO	Trạng thái đã được xác nhận; fail khi quá thời gian chờ.
created_at	DATETIME	INDEX	NO	Thời điểm gửi lệnh.
confirmed_at	DATETIME	—	YES	Thời điểm ESP32 xác nhận; NULL nếu loading hoặc fail.

4.7. Quy tắc dữ liệu quan trọng

- action ghi nhận mong muốn điều khiển do người dùng gửi đi; status ghi nhận trạng thái đã được ESP32 xác nhận. Hai trường không được đồng nhất.
- Khi vừa gửi lệnh, status trong ActionHistory bắt buộc là loading; chỉ chuyển sang on/off sau khi nhận phản hồi qua topic iot/devices/status từ ESP32, hoặc sang fail khi quá 15 giây không có phản hồi.
- Các khóa chính id dùng BIGINT, AUTO_INCREMENT.

- created_at và measured_at nên được đánh index (kết hợp sensor_id, device_id hoặc user_id khi cần) để hỗ trợ lọc theo thời gian ở các màn hình Data Sensor và Action History.
- Mật khẩu chỉ lưu hash_password, không lưu mật khẩu dạng rõ.

Chương 5: KẾT QUẢ VÀ ĐÁNH GIÁ THỰC NGHIỆM

5.1. Cấu trúc thư mục dự án

```
D:\IoT
├── esp32_iot/esp32_iot.ino      # firmware ESP32
├── esp32_hwtest/esp32_hwtest.ino # test phần cứng không cần Wi-Fi
├── setup.ps1, iot.ps1            # cài Broker, lệnh tắt Pub/Sub/Led
├── mosquitto.conf, passwd        # cấu hình Broker cổng 8899
├── iot-backend/                  # Nuxt 4 (Nitro)
│   ├── server/api/               # REST API
│   ├── server/plugins/mqtt.ts    # MQTT client
│   ├── server/routes/realtime.ts # WebSocket
│   ├── server/middleware/        # CORS, JWT
│   ├── server/utils/             # db, sensors, devices, actions...
│   ├── database/schema.sql
│   └── scripts/                  # db-init, esp-simulator
└── iot-frontend/                 # Vue 3 + Vite
    └── src/ views/ components/ stores/ services/ router/
```

5.2. Kết quả triển khai phần cứng

- Bo ESP32 NodeMCU-32S (chip ESP32-D0WD-V3) được nạp firmware esp32_iot.ino bằng Arduino IDE (board ESP32 Dev Module); DHT11 nối GPIO 4, module LDR nối GPIO 34, ba đèn LED nối GPIO 5, 18, 19. Trước đó phần cứng được kiểm tra riêng bằng chương trình esp32_hwtest.ino (không cần Wi-Fi): DHT11 đọc 29.8 °C / 50.3 %, LDR có tín hiệu, cả 3 đèn sáng đúng thứ tự.
- ESP32 kết nối Wi-Fi 2.4 GHz, đồng bộ giờ với pool.ntp.org (GMT+7), rồi kết nối Broker tại IP của laptop cổng 8899 bằng tài khoản MQTT với client ID ESP32_IOT; khi mất kết nối, firmware thử lại sau mỗi 2 giây và subscribe lại iot/devices/control.
- Mỗi 2 giây firmware gửi một bản tin iot/sensors/data gồm timestamp, nhiệt độ, độ ẩm và ánh sáng (lux). Khi đọc DHT11 lỗi, bản tin vẫn được gửi nhưng thiếu hai trường nhiệt độ, độ ẩm.

[Chèn ảnh chụp mạch ESP32 thực tế và ảnh Serial Monitor tại đây — Hình 5.1]

5.3. Kết quả triển khai Broker và Backend

Broker Mosquitto chạy bằng lệnh mosquitto -c D:\IoT\mosquitto.conf -v, mở cổng 8899 và từ chối mọi kết nối không có tài khoản. Kiểm thử bằng Mosquitto CLI:

Lắng nghe dữ liệu cảm biến:

```
mosquitto_sub -h localhost -p 8899 -u HoangManhHiep -P B23DCCN292 -t
iot/sensors/data -v
```

Giả lập phần cứng gửi một bản tin cảm biến:

```
mosquitto_pub -h localhost -p 8899 -u HoangManhHiep -P B23DCCN292 -t
iot/sensors/data -m '{"timestamp":"2026-11-10
09:30:00","temperature":28.5,"humidity":60,"light":320}'
```

Gửi lệnh bật đèn 1 và lắng nghe phản hồi:

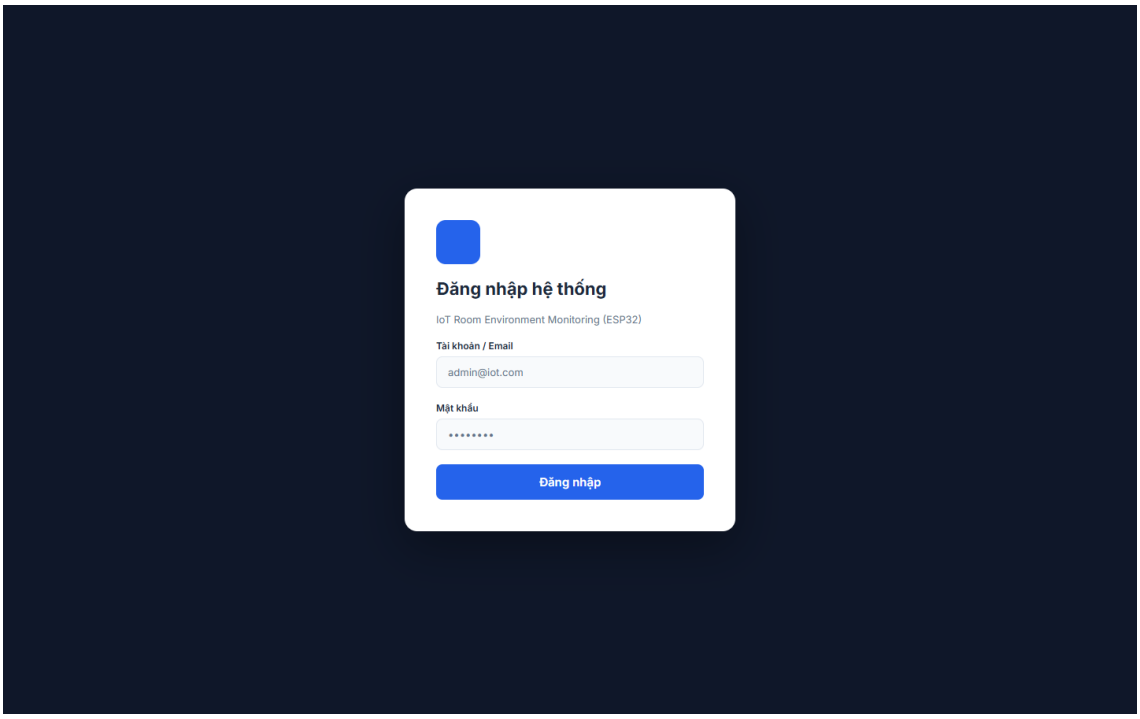
```
mosquitto_pub -h localhost -p 8899 -u HoangManhHiep -P B23DCCN292 -t
iot/devices/control -m LED1_ON
mosquitto_sub -h localhost -p 8899 -u HoangManhHiep -P B23DCCN292 -t
iot/devices/status -v
```

Backend Nuxt chạy tại http://localhost:3000:

- npm run db:init tạo database iot_monitor, 5 bảng, 2 tài khoản demo (mật khẩu băm bcrypt), 3 cảm biến và 3 đèn; npm run db:sample sinh thêm 24 giờ dữ liệu mẫu.
- Khi khởi động, plugin MQTT kết nối Broker và subscribe iot/sensors/data, iot/devices/status; GET /api/health trả về database: true, mqtt: true.
- Khi Broker bị tắt, health chuyển sang mqtt: false và API điều khiển trả 503; khi Broker chạy lại, Backend tự kết nối lại (chu kỳ 3 giây) mà không cần khởi động lại.
- Mỗi bản tin iot/sensors/data được ghi thành 3 bản ghi sensor_data (nhiệt độ, độ ẩm, ánh sáng) có chung measured_at lấy từ timestamp NTP, rồi đẩy sự kiện sensor:update qua WebSocket tới mọi trình duyệt đang mở.

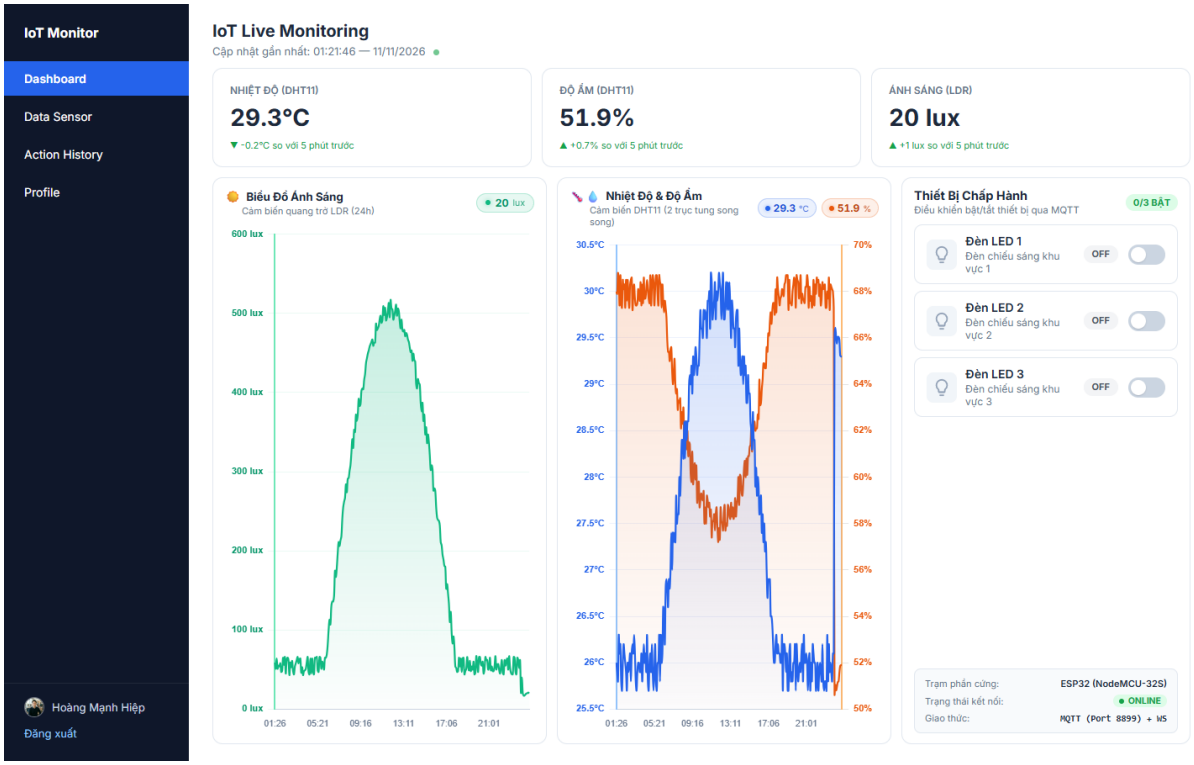
5.4. Kết quả xây dựng giao diện Frontend

Màn hình Đăng nhập: form tài khoản/mật khẩu, báo lỗi khi sai thông tin; đăng nhập thành công chuyển tới Dashboard.



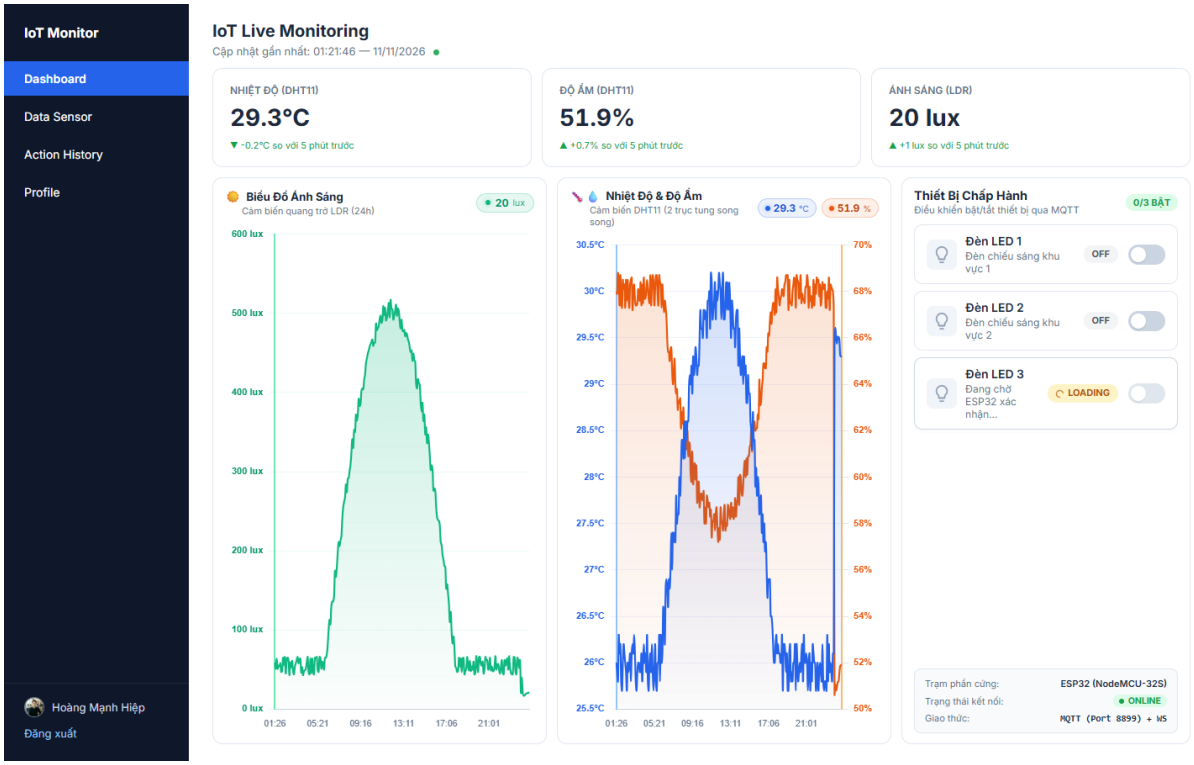
Hình 5.2: Màn hình đăng nhập (chạy thực tế)

Dashboard: ba ô số liệu nhiệt độ, độ ẩm, ánh sáng kèm chênh lệch so với 5 phút trước; biểu đồ ánh sáng và biểu đồ nhiệt độ – độ ẩm (2 trục tung) trong 24 giờ; khối Thiết bị chấp hành gồm 3 đèn kèm trạng thái kết nối của trạm ESP32.

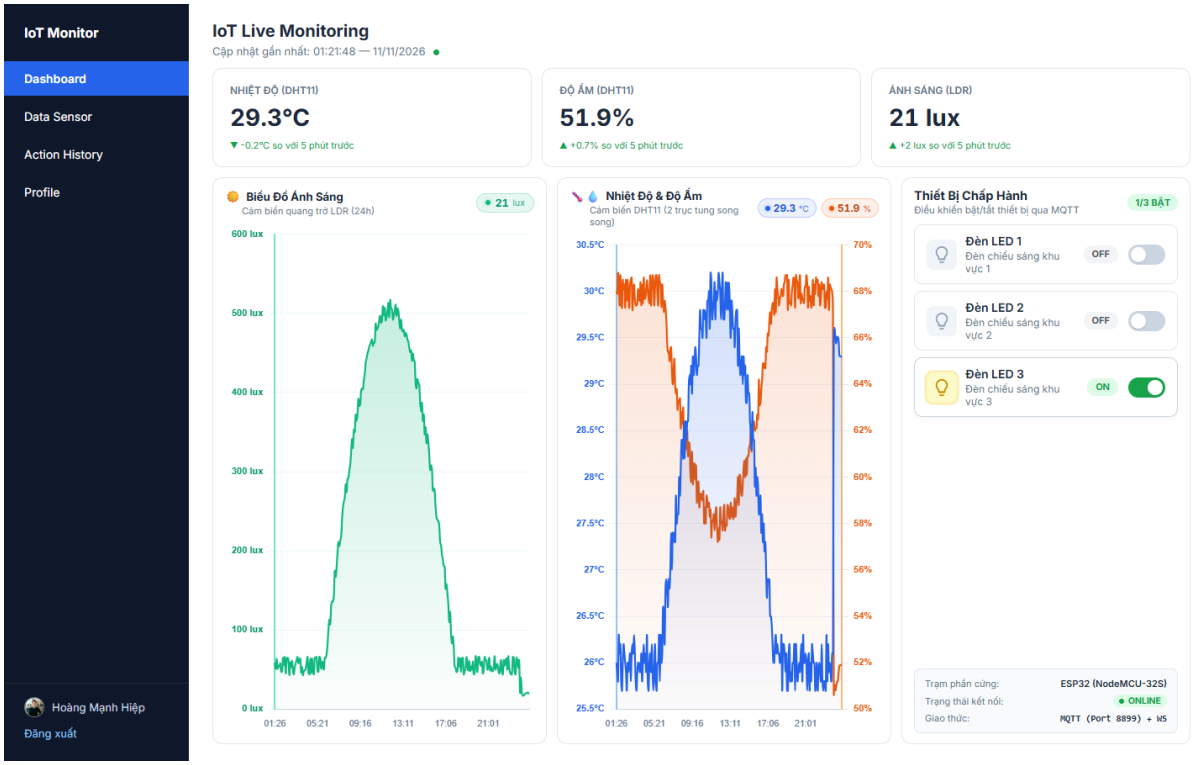


Hình 5.3: Màn hình Dashboard

Điều khiển đèn: khi gạt công tắc, đèn chuyển sang LOADING ("Đang chờ ESP32 xác nhận...") và chỉ chuyển ON/OFF khi nhận được bản tin trên iot/devices/status.

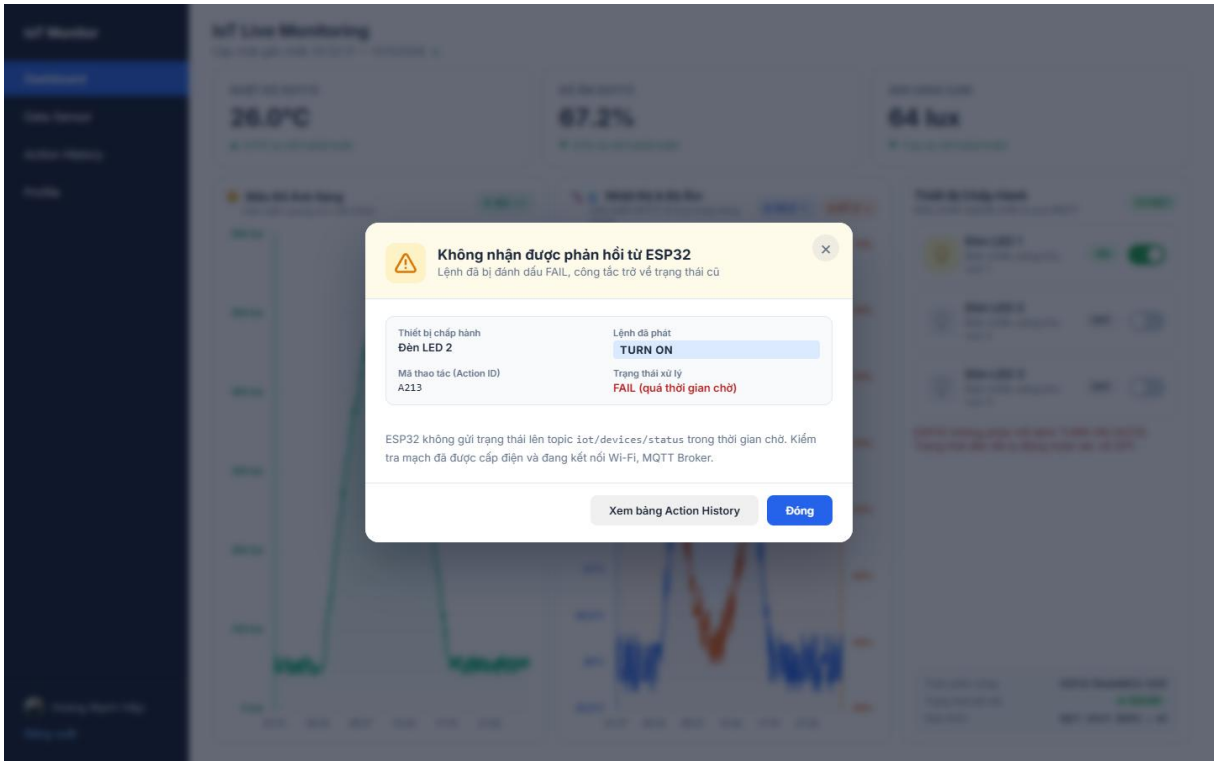


Hình 5.4: Đèn LED 3 ở trạng thái LOADING chờ xác nhận



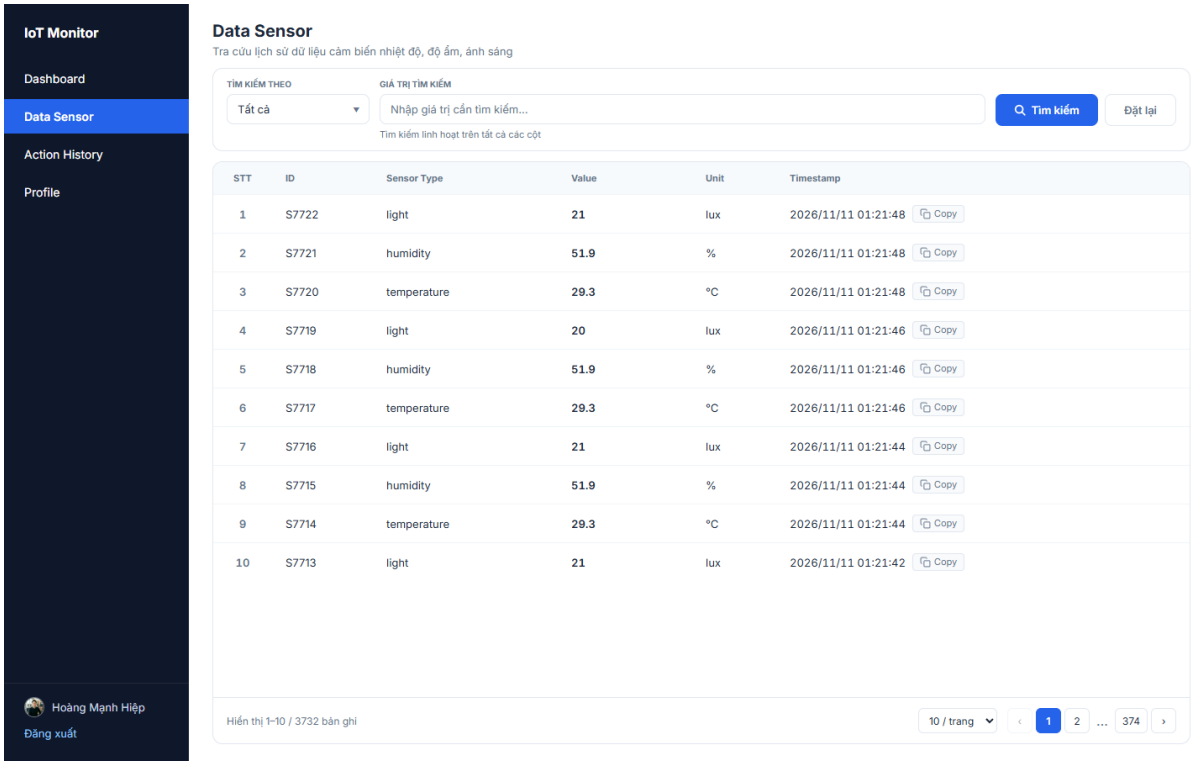
Hình 5.5: Đèn LED 3 chuyển sang ON sau khi ESP32 xác nhận

Ngoại lệ không nhận phản hồi: khi ESP32 không phản hồi lệnh, sau thời gian chờ giao diện hiện hộp thoại ngoại lệ, công tắc trở về trạng thái cũ và bản ghi được đánh dấu FAIL.



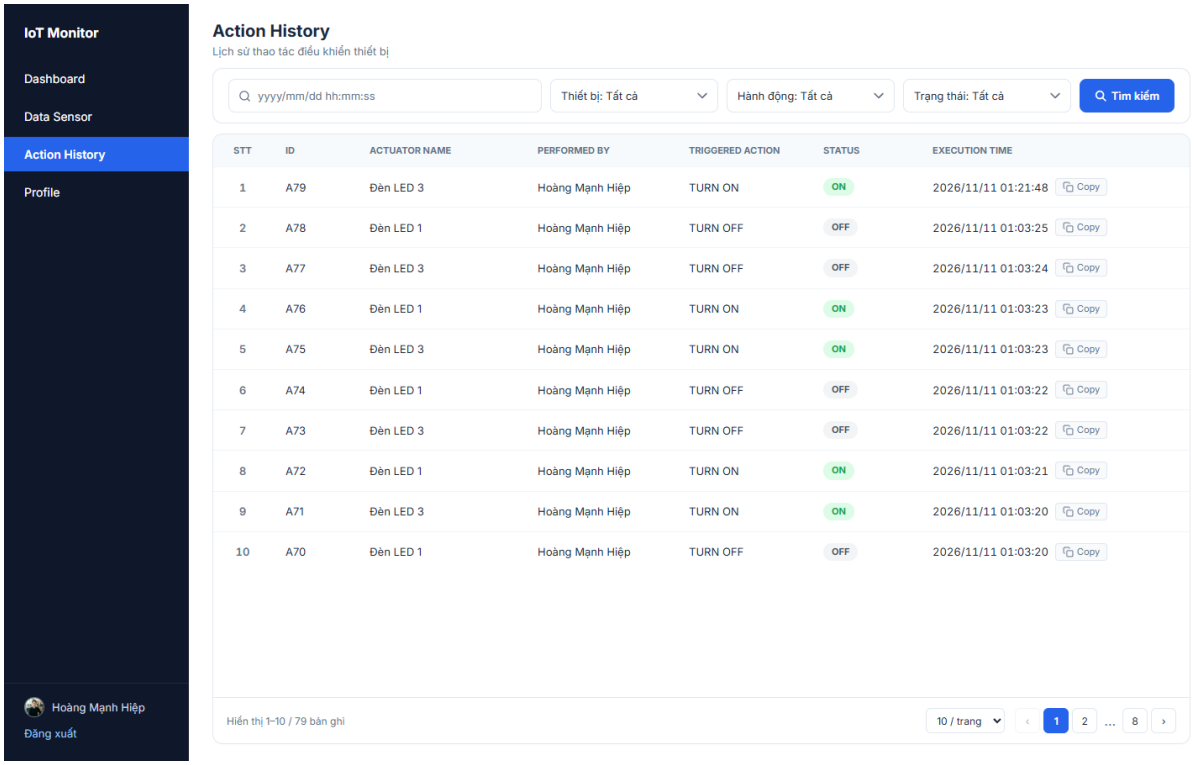
Hình 5.6: Hộp thoại ngoại lệ khi lệnh quá thời gian chờ

Data Sensor: bảng lịch sử có lọc theo loại cảm biến, ô tìm kiếm theo giá trị hoặc thời gian yyyy/mm/dd hh:mm:ss (cho phép nhập thiếu để mở rộng phạm vi), phân trang.



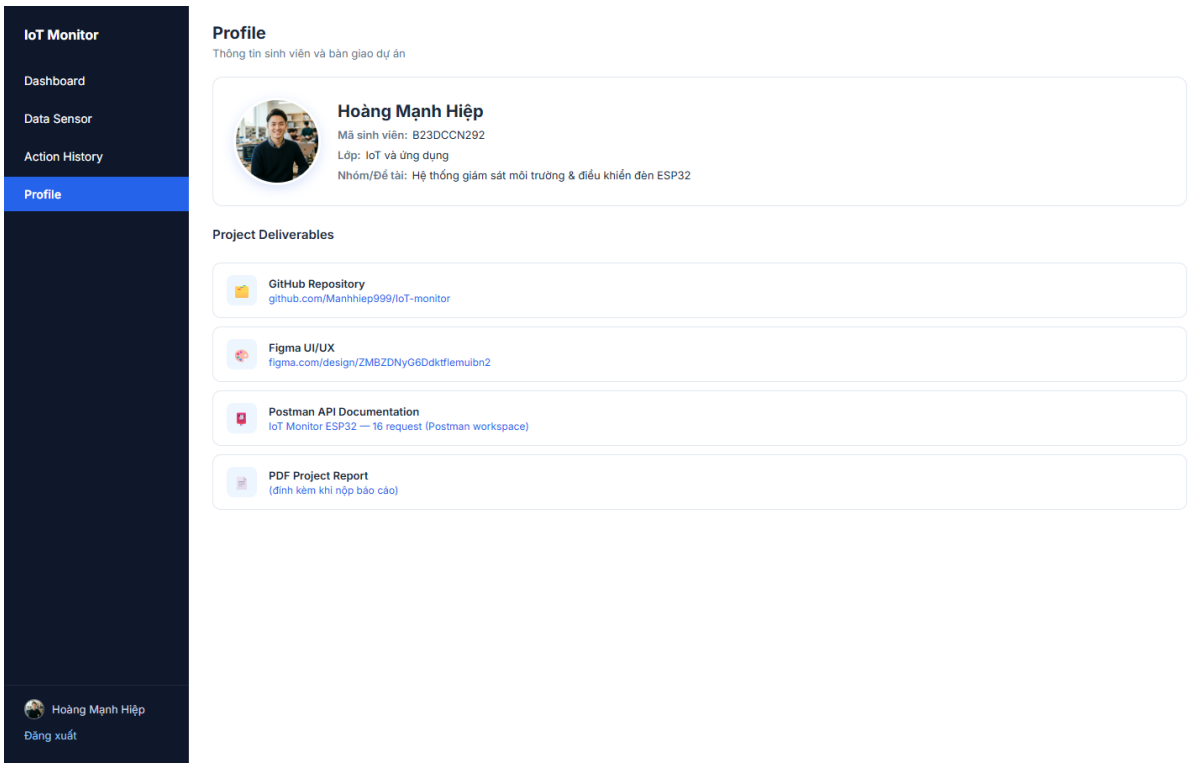
Hình 5.7: Màn hình Data Sensor

Action History: lịch sử thao tác với các cột ID, tên đèn, người thực hiện, hành động, trạng thái (ON/OFF/LOADING/FAIL), thời gian; lọc theo thiết bị, hành động, trạng thái, thời gian; phân trang.



Hình 5.8: Màn hình Action History

Profile: thông tin sinh viên và các liên kết GitHub, Figma, Postman, báo cáo PDF.



Hình 5.9: Màn hình Profile

5.5. Đánh giá thử nghiệm và kiểm thử hệ thống

Các phép đo dưới đây được thực hiện trên laptop chạy đồng thời Mosquitto, Backend, MySQL và Frontend, với ESP32 được thay bằng chương trình giả lập esp-simulator.mjs (cùng topic và payload với firmware, phản hồi ngay không trễ). Vì vậy số liệu phản ánh độ trễ xử lý của Broker – Backend – WebSocket, chưa gồm thời gian truyền Wi-Fi và xử lý trên ESP32 thật:

Chỉ số	Số lần đo	Nhỏ nhất	Trung bình	Lớn nhất
Điều khiển: từ lúc gửi POST /actions đến khi nhận device:status	20	13,7 ms	23,7 ms	59,9 ms
Giám sát: từ lúc publish iot/sensors/data đến khi nhận sensor:update	30	8,9 ms	10,2 ms	13,7 ms

Đo trên mạch ESP32 thật (ngày 11/11/2026; ESP32 và laptop cùng hotspot Wi-Fi 2.4 GHz, Broker – Backend – MySQL – Frontend chạy trên laptop):

Chỉ số	Kết quả đo
Thời gian chạy liên tục	17 phút (00:49:26 – 01:06:28, tổng 1022 giây)
Bản tin iot/sensors/data đúng ra phải nhận (2 giây/lần)	1022 ÷ 2 + 1 = 512 bản tin
Bản tin thực nhận trong bảng sensor_data	512 bản tin — tỷ lệ mất gói 0 %, không có khoảng trống nào dài hơn 3 giây
Lệnh điều khiển gửi từ giao diện web	72 lệnh, 72/72 được ESP32 xác nhận (0 lệnh FAIL)
Độ trễ khứ hồi điều khiển (POST /actions → đèn đổi trạng thái → nhận device:status), 14 lần đo	Nhỏ nhất 22 ms; trung bình 105 ms; trung vị 106 ms; lớn nhất 197 ms

Độ trễ trên mạch thật cao hơn khi dùng ESP32 giả lập (khoảng 105 ms so với 24 ms) vì có thêm thời gian truyền qua Wi-Fi và thời gian ESP32 xử lý lệnh, nhưng vẫn dưới 200 ms nên với người dùng, đèn gần như phản hồi tức thì. Trong 17 phút chạy liên tục, hệ thống không mất bản tin nào và không có lệnh điều khiển nào thất bại.

Bảng kiểm thử chức năng:

Trường hợp kiểm thử	Mô tả	Kết quả mong đợi	Kết quả thực tế
Nhận dữ liệu cảm biến	Publish iot/sensors/data mỗi 2 giây (timestamp + 3 giá trị)	Lưu 3 bản ghi/lần theo giờ NTP, Dashboard cập nhật không tải lại trang	Đạt
Bật/tắt đèn có xác nhận	Bật LED 1, tắt LED 3 trên Dashboard	LOADING → ON/OFF, ActionHistory có confirmed_at	Đạt
Không nhận phản hồi	Giả lập ESP32 không trả lời lệnh của LED 2	Sau thời gian chờ: status = fail, công tắc về trạng thái cũ, hiện hộp thoại	Đạt
Gửi lệnh khi đang chờ	Gửi lệnh thứ hai cho LED 2 khi còn loading	HTTP 409, không tạo bản ghi mới	Đạt
Dữ liệu đầu vào sai	action = "X"; dateFrom = "abc"	HTTP 400 kèm thông báo lỗi	Đạt
Đăng nhập sai / thiếu token	Sai mật khẩu; gọi API không có token	HTTP 401	Đạt
Mất kết nối Broker	Tắt Mosquitto rồi bật lại	health.mqtt = false, sau đó Backend tự kết nối lại	Đạt
Tra cứu lịch sử	Lọc nhiệt độ chứa "28"; lọc thời gian theo giờ; lọc status = fail	Trả đúng bản ghi, có tổng số và phân trang	Đạt
Xuất CSV (API)	Gọi GET /api/devices/actions/export ?action=ON	File text/csv UTF-8 mở được bằng Excel	Đạt
Thời gian NTP lệch / N/A	Gửi timestamp 2020-01-01, "N/A" và giờ lệch 4 phút	Lệch nhiều hoặc N/A dùng giờ máy chủ; lệch ≤ 5 phút giữ giờ ESP32	Đạt
Phản hồi ngược lệnh	Gửi lệnh LED2 ON, giả lập ESP32 trả LED2 OFF	Bản ghi chuyển FAIL, đèn hiển thị đúng trạng thái thật (OFF)	Đạt
Thiếu DHT11	Bản tin không có temperature/humidity	Lưu các giá trị còn lại, không báo lỗi	Đạt

5.6. Triển khai hệ thống

Thành phần	Cấu hình triển khai
MQTT Broker (Mosquitto)	Chạy trên laptop, cổng 8899, allow_anonymous false, tài khoản HoangManhHiep.
Backend Nuxt	http://localhost:3000 (npm run dev / npm run build && npm start), cấu hình qua .env: NUXT_MQTT_URL, NUXT_DB_*, NUXT_JWT_SECRET, NUXT_ACTION_TIMEOUT_SECONDS=15.
MySQL	127.0.0.1:3306, database iot_monitor, 5 bảng users, sensors, devices, sensor_data, action_history.
Frontend Vue (Vite)	http://localhost:5173, VITE_USE_MOCK=false, proxy /api và /realtime tới Backend.
ESP32 NodeMCU-32S	Firmware esp32_iot/esp32_iot.ino (Wi-Fi, MQTT trong secrets.h), Wi-Fi 2.4 GHz, mqtt_server là IPv4 của laptop, NTP pool.ntp.org; topic iot/sensors/data / iot/devices/control / iot/devices/status.

Chương 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Đề tài đã hoàn thành hệ thống giám sát nhiệt độ, độ ẩm, ánh sáng và điều khiển đèn LED phòng học bằng ESP32, từ thiết kế (5 màn hình Figma, sáu Use Case, sáu Sequence Diagram, lược đồ cơ sở dữ liệu 5 bảng, tài liệu API và quy ước MQTT) đến hiện thực firmware, Backend Nuxt.js, Frontend Vue.js và kiểm thử. Xuyên suốt các phần, hệ thống giữ nhất quán nguyên tắc phân biệt giữa action người dùng yêu cầu và status thật đã được phần cứng xác nhận, thể hiện rõ nhất qua UC04 và cột confirmed_at trong bảng ActionHistory.

Các kết quả chính:

- Kết nối liền mạch các tầng: ESP32 + DHT11 + LDR + 3 LED → Mosquitto (MQTT có xác thực) → Backend Nuxt → MySQL → Frontend Vue qua REST API và WebSocket.
- Điều khiển hai chiều có xác nhận: trạng thái LOADING trong lúc chờ, chỉ chốt ON/OFF theo trạng thái đọc lại từ chân GPIO; tự đánh dấu FAIL và hoàn tác giao diện khi quá thời gian chờ.
- Lưu trữ và tra cứu lịch sử cảm biến, lịch sử thao tác với bộ lọc thời gian linh hoạt và phân trang; API hỗ trợ xuất lịch sử thao tác ra CSV.

6.1. Các hạn chế còn tồn tại

- DHT11 có dải đo hẹp và độ chính xác thấp (±2 °C, ±5 %RH); giá trị lux của LDR được quy đổi theo đặc tuyến gần đúng (R10 = 15 kΩ, γ = 0.7), chưa hiệu chuẩn bằng máy đo lux.
- Thời điểm đo phụ thuộc đồng bộ NTP của ESP32: nếu mạng không ra được Internet, ESP32 gửi timestamp "N/A" và Backend phải dùng giờ máy chủ lúc nhận, nên dữ liệu gửi trễ sẽ mang mốc thời gian lúc nhận.
- Lệnh và phản hồi trạng thái chưa kèm mã lệnh (actionId), Backend ghép phản hồi với lệnh đang chờ theo tên đèn; nếu hai người cùng điều khiển một đèn gần như đồng thời, việc ghép phản hồi với lệnh có thể chưa chính xác.
- Kết nối MQTT, API và WebSocket chưa mã hóa (chưa dùng TLS/HTTPS/WSS); hệ thống mới quản lý một phòng học, chưa phân quyền nhiều vai trò; mới điều khiển LED công suất nhỏ, chưa dùng relay cho thiết bị 220 V.

6.2. Hướng phát triển

- Thay DHT11 bằng SHT31/BME280, thay LDR bằng cảm biến ánh sáng số BH1750 để đo lux chính xác; bổ sung cảm biến CO_2, bụi PM2.5, chuyển động PIR.
- Gửi kèm actionId trong lệnh và phản hồi trạng thái để xác nhận lệnh chính xác tuyệt đối; bổ sung cảm biến khí gas (MQ-2), bụi mịn (PMS5003), tiếng ồn (KY-037) kèm cảnh báo vượt ngưỡng — cột warn_above trong bảng Sensors đã được dự phòng cho việc này.
- Dùng relay cách ly quang để điều khiển đèn, quạt thật; thêm luật tự động (bật đèn khi trời tối, cảnh báo khi nhiệt độ vượt ngưỡng).
- Bật MQTT over TLS (cổng 8883), HTTPS/WSS; hỗ trợ nhiều phòng học, phân quyền quản trị viên/người dùng; ứng dụng di động có thông báo đẩy khi chỉ số vượt ngưỡng.